

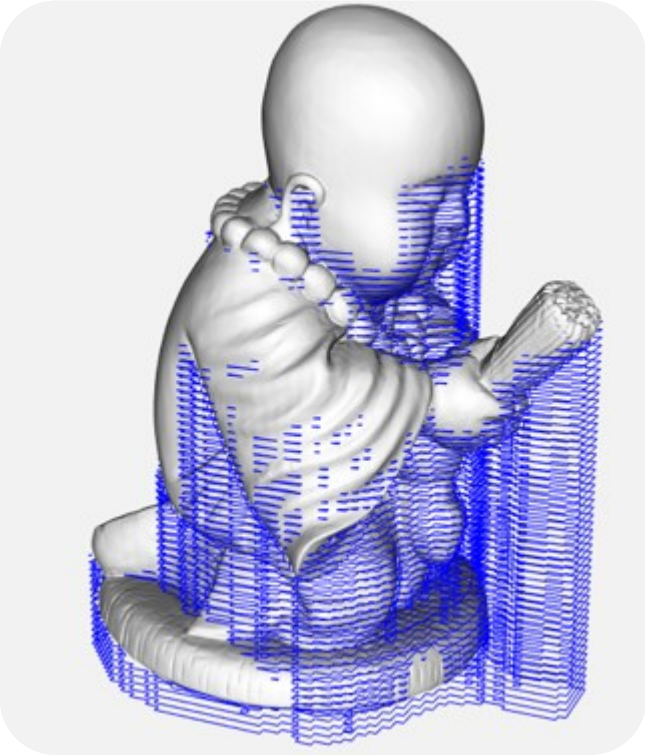
计算机辅助制造实践

——Python实现三维打印路径规划

浙江大学机械工程学院 林志伟
玉泉教1-215　zjtzhylin1986@163.com

第10章 支撑生成与优化

- 支撑生成概述
- 支撑区域识别
- 支撑路径生成
- 支撑量优化



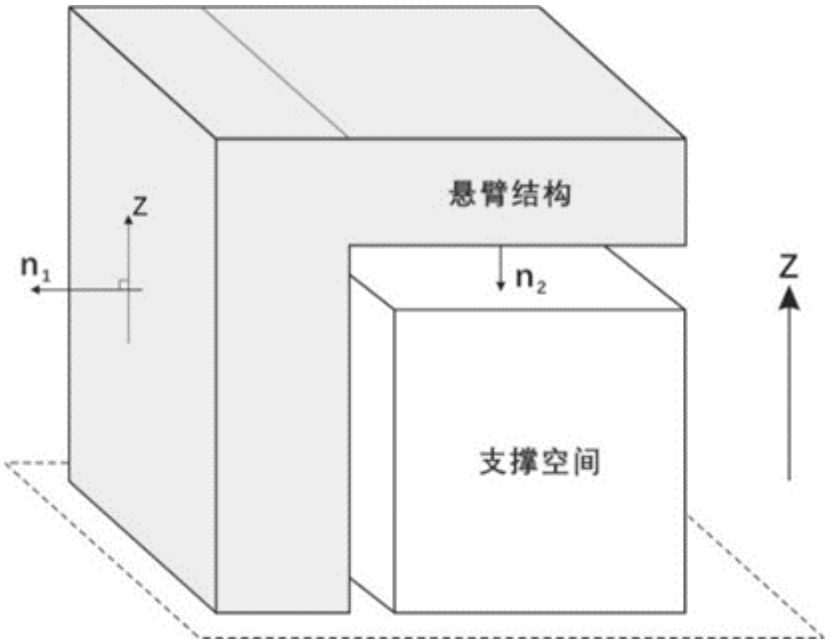
1. 支撑生成概述

三维打印是一种典型的**增材制造**过程，根据其成形工艺特点，在工件从无到有的变化过程中，如果遭遇尺寸较大的**悬臂结构**，即在某层切片下方没有上一层切片材料为其**托底**，打印将无法继续进行。因此，如果模型上存在悬臂结构，必须考虑在悬臂下方**添加支撑材料**，并生成相应的**支撑填充路径**。

问题：

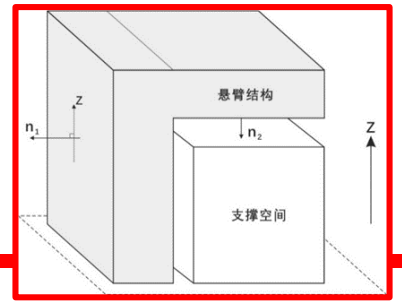
① 如何**识别**模型上的**悬臂结构**？是否只有在水平的悬臂结构下才需要添加支撑？

② 在得到了需要支撑的空间之后又该如何**生成支撑路径**？

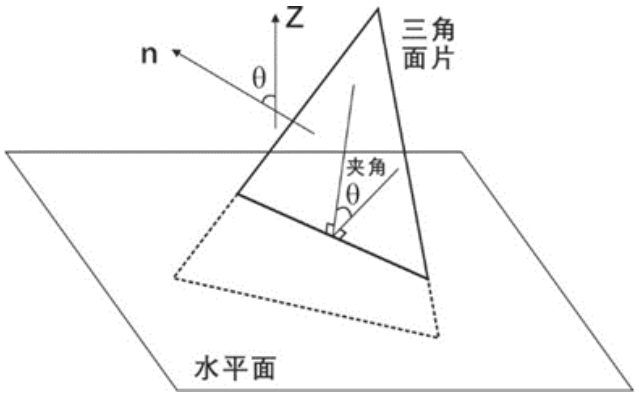


三维模型上的悬臂结构

1. 支撑生成概述



- **危险面片寻找**
- **悬臂特征：**直观上，添加支撑的**首要任务**是**找出模型上的悬臂特征。**
  - 但对大多数三维模型文件格式来说，一般只保存模型的点、线、面等基础几何信息，而不保存模型内部复杂的逻辑特征信息。
  - 如对STL格式，它保存的就是三角面片的顶点坐标和法向量信息，且每个面片无序排放的。**在这种情况下，必须借助相应算法，对模型进行分析，才能找出模型上的悬臂特征。**
- **悬臂面片：**当模型中面片和水平面**垂直**时，无需为其添加支撑，称这种面片为**非悬臂面片**或**安全面片**；相反地，当模型中面片和水平面**平行**时，需要为其添加支撑，称这种面片为**悬臂面片**或**危险面片**。
  - 原则上，只需计算STL模型中所有**三角面片**和**水平面**的夹角，就能找出所有需要添加支撑的危险面片。
  - 进一步地，如图，面片和水平面的夹角等同于面片法向量和竖直方向（即坐标系Z轴方向）的夹角（记为θ），因此，可通过计算面片法向量和竖直方向来确定模型上的危险面片。



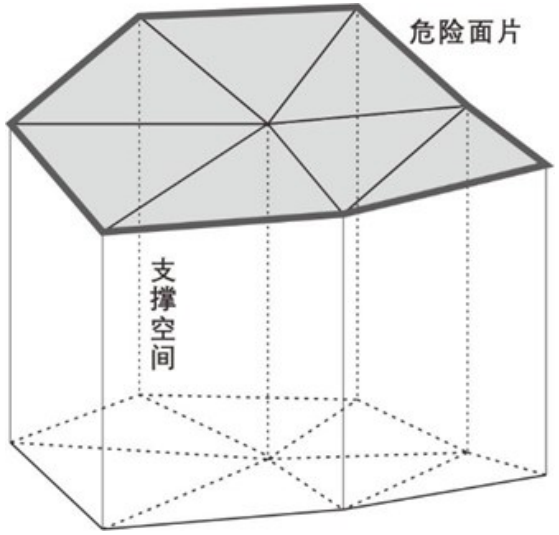
三角面片和水平面角度计算

✓ 在实际三维打印应用中，在0到90度范围内，θ值越大，面片越安全，θ值越小，面片越危险。

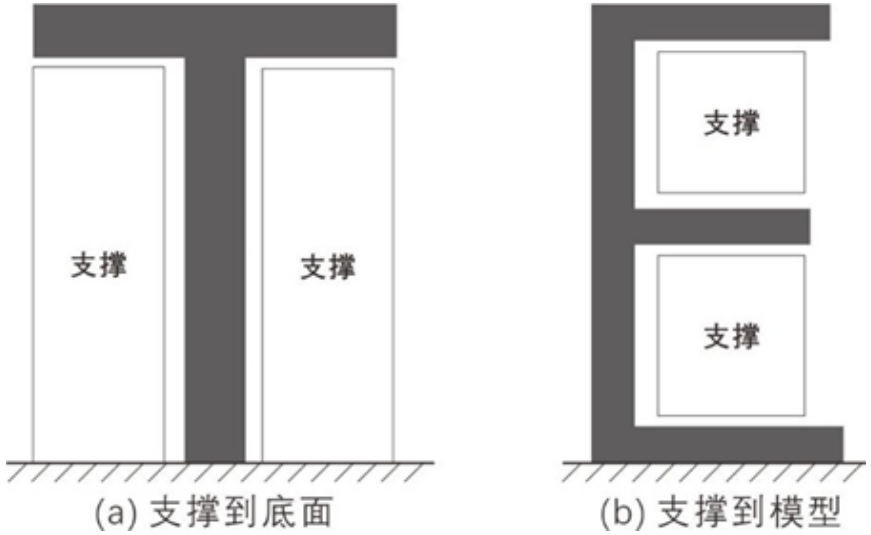
✓ **支撑临界角定义：**实际计算中，一般会根据三维打印的工艺类型及材料属性为θ设定一个临界值θ_0（如60度），并认为：当实际$\theta > \theta_0$时，面片为安全面片；当$\theta \leq \theta_0$时，面片为危险面片。

1. 支撑生成概述

- **危险面片支撑空间**
- 确定模型中的所有**危险面片**后，下一步需要在这些**危险面片**的下方**确定需要添加支撑的空间区域**。
- 对每个危险面片来说，它的支撑区域就是位于其下方竖直投影空间内的**三棱柱**。
  - 如左图所示，其中三棱柱的上端面就是危险面片本身，而下端面可能立于工作台面，也可能位于模型上，分别对应三维打印的两种支撑类型：**支撑到底面（即工作台）**或**支撑到模型**，如右图。
- 如果不加处理，直接对每个危险面片使用竖直投影法生成**支撑空间**，那最终将得到一系列瘦长的三棱柱（左图）。
  - **希望**：相邻的离散三棱柱空间最好能够连成一体，得到尺寸相对较大的成片支撑空间，以便在其内部生成支撑路径。



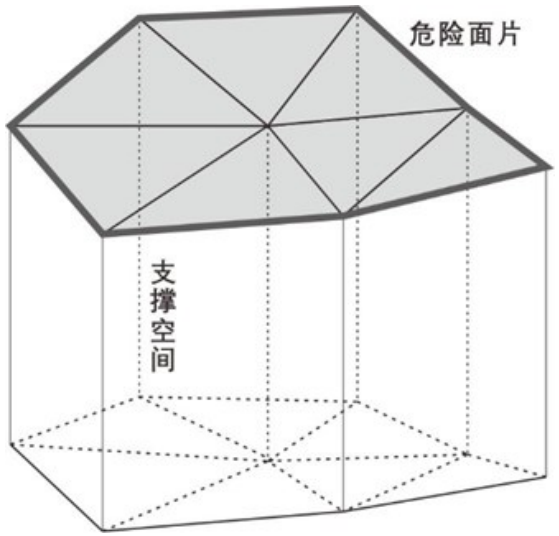
危险面片形成的三棱柱支撑空间



支撑下端面接触情况：支撑到底面或模型

1. 支撑生成概述

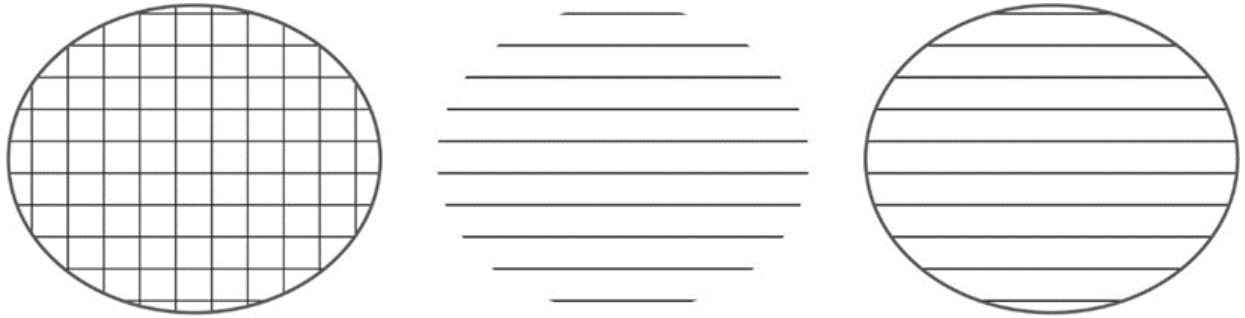
- **如何生成模型支撑空间?**
- **可以在三维空间中对得到三棱柱进行拼接操作，但更容易操作的方式是先对得到的危险三角面片进行拼接。**
  - 考虑到STL模型中三角面片存储的**无序性**，可仿照第七章介绍的方法，首先**建立各个危险面片之间的拓扑索引关系**，然后进一步组织并将所有存在相邻关系（共享公共边）的危险面片连成一片。
  - 在此基础上，对每组已经连接成片的危险面片，可以提取其边界，形成一条封闭的空间多边形（也可能是若干条存在包含关系的空间多边形），如图中粗线。
  - 对得到的空间多边形，理论上，只要我们对其进行向下竖直投影，就能生成所求的支撑空间。



危险面片形成的三棱柱支撑空间

1. 支撑生成概述

- **如何生成支撑路径?**
- **支撑空间切片：对得到的支撑空间区域，需要在其内部生成支撑路径，首先是切片。**
  - 将支撑空间看成一个实体模型，在对STL模型进行切片时，可以将支撑空间作为模型的一部分一起进行切片，生成模型切片轮廓和支撑切片轮廓。
  - 注意：由于切片同时进行，因此支撑切片每一层在Z向的高度和模型切片相应层在Z向的高度保持一致，在每层切片打印过程中，可以让**喷头保持相同的高度**去打印模型材料和支撑材料。
- **支撑路径生成：对得到的支撑切片轮廓，下一步在其内部生成填充路径。**
  - 可使用轮廓和平行填充，其中轮廓路径用于外围填充，平行路径用于内部填充。考虑到支撑部分对轮廓本身精度不做要求，因此轮廓路径可以少用甚至不用。
  - 如图a，对相邻上下两层切片，可设置其填充角度差为 0 度或 90 度。当填充角度差为90度（交叉填充）时，上下两层填充材料可形成编织效果，支撑不易倒塌。但需要注意的是，当填充角度差为0度（同向填充）时，如图b和c所示，有必要在平行路径外围添加若干圈轮廓路径以加固支撑，否则支撑容易倒塌。
  - 相对于STL模型部分的填充路径，支撑部分的填充路径可以设置得更为**稀疏**，这样做一方面可以节省打印材料和打印时间，另一方面也可降低支撑强度，使其更容易从工作台或模型上剥离。



(a) 轮廓+平行交叉填充　(b) 平行同向填充　(c) 轮廓+平行同向填充

支撑打印填充路径组织方式

2. 支撑区域识别

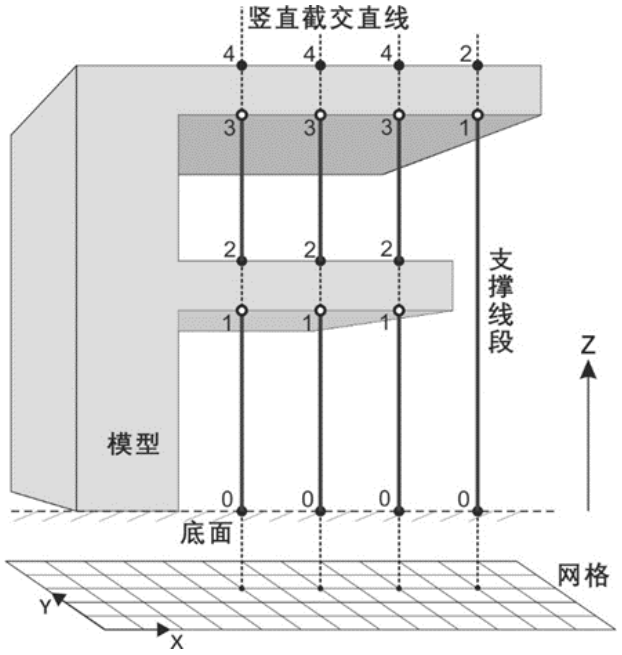
- 前面以一种容易理解的方式解释了三维模型上支撑区域的识别方法，即：

遍历找出模型上所有危险面片 → 合并相邻危险面片 → 对合并面片竖直投影得到需要添加支撑的空间区域

- 以上方法虽然原理简单，但**实施难度极大**，如设计面片合并算法、面片投影算法等都可能需要耗费较大精力。

一种更容易实施的支撑区域生成方法，其核心思想：

- 使用一条**竖直直线**去截交三维模型及其底面（指过模型最低点的一张水平面），并对得到的交点沿直线方向从下到上排序；
- 直观地，位于偶数和奇数序号交点之间的线段可以看作是模型在此处需要添加的支撑，不妨称这种线段为**支撑线段**。
- 理论上，只要无限增加直线数量，就可找出模型上的所有支撑线段，从而准确地确定模型上需要添加支撑的空间区域。
  - 在实际操作中，无限增加截交直线的数量是不可能的，但可以采用**离散**思想，在模型所在的**XY平面范围**内**生成均匀网格**，并在**网格节点处**布置竖直截交直线，这样就可以通过控制平面网格尺寸的方式控制截交直线的密度，从而控制支撑区域的精度。



基于直线截交的三维模型支撑识别原理

0到1号、2到3号点之间的线段为**支撑线段**

2. 支撑区域识别

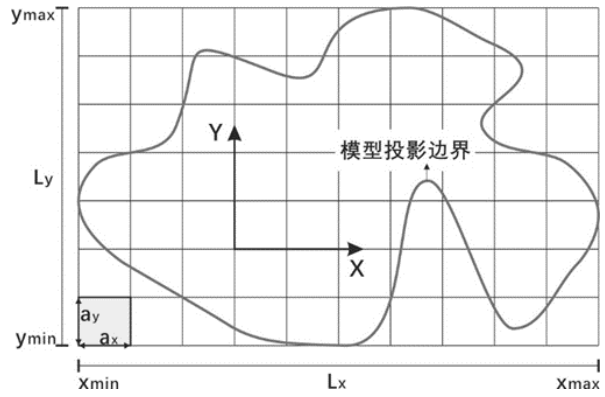
• 2.1. 模型投影网格生成

- **包络矩形获取：**对任一给定STL模型，调用已实现的 **getBounds** 函数可获取模型在 X、Y 两个方向上的极限坐标 x_{min}、x_{max} 以及 y_{min}、y_{max}。在XY平面上，构建模型轴平行包络矩形，注意，这里轴平行是指矩形的各条边分别和坐标轴平行。
- **网格划分：**简单起见，预设网格形状为**正方形**，且其边长为a。网格划分完成后，所有网格恰好能够覆盖整个包络矩形。记包络矩形在X、Y方向的边长分别为 L_x、L_y，考虑到实际很难保证 L_x 和 L_y 恰好都是 a 的倍数，有必要对网格边长进行微调。具体地，以预设边长 a 为基础，可得调整后边长（记为a_x、a_y）为：

$$a_x = \frac{L_x}{\mathrm{int}(L_x/a)+1}, \quad (10\text{-}1)$$

$$a_y = \frac{L_y}{\mathrm{int}(L_y/a)+1}. \quad (10\text{-}2)$$

- 式中int为取整符号，如int(7.6) = 7，int(8.1) = 8，即截取实数的整数部分；
- 分母加1的目的是保证得到的边长a_x小于预设边长a。



基于直线截交的三维模型支撑识别原理

- 结合 x_{min}、x_{max}、y_{min}、y_{max} 以及调整网格边长后得到的 a_x、a_y，可以计算图中各网格节点坐标。
- 如在X方向上，各节点X坐标依次为：x_{min}、$x_{min}+a_x$、$x_{min}+2a_x$、…、x_{max}；
- 而在Y方向上，各节点Y坐标依次为：y_{min}、$y_{min}+a_y$、$y_{min}+2a_y$、…、y_{max}。
- 只要对以上x、y序列中各值两两组合，就可以生成网格中所有节点坐标。

2. 支撑区域识别

• 2.1. 模型投影网格生成：代码实现

- 建立 **FindSptRegion 类**，其包含 gridSize、crAngle 等属性，实现 **initGrids** 函数如下：

FindSptRegion.py | 寻找支撑区域类，初始化和三维模型平面网格生成 | 10-1

```
1   class FindSptRegion:
2     def __init__(self, stlModel, layers, gridSize, crAngle, xyGap = 1):  # 类初始化
3       self.digit = 3                                  # 计算精度，默认保留3位小数
4       self.stlModel = stlModel                        # 保存三维模型引用
5       self.layers = layers                            # 保存切片数据引用
6       self.gridSize = gridSize                        # 预设正方形网格边长，即a值
7       self.ax, self.ay = 0, 0                         # 存储调整后的网格边长ax、ay
8       self.crAngle= crAngle                           # 支撑临界角，即θ0
9       self.xyGap = xyGap                              # 支撑到模型的横向间隙
10    def execute(self):                                # 类的核心函数，具体实现见后文
11      pass
12    def initGrids(self):                              # 定义网格生成函数
13      xMin, xMax, yMin, yMax, zMin, zMax = self.stlModel.getBounds()      # 模型各方向极限
14      self.ax = (xMax - xMin) / ((int)((xMax - xMin)/self.gridSize) + 1)  # 网格X方向边长
15      self.ay = (yMax - yMin) / ((int)((yMax - yMin)/self.gridSize) + 1)  # 网格Y方向边长
16      xs, ys, gridDic, x, y = [], [], {}, xMin, yMin  # 定义临时列表、字典变量等
17      while x <= xMax:                                # 生成X方向网格坐标序列
18        xs.append(round(x, self.digit))
19        x += self.ax
20      while y <= yMax:                                # 生成Y方向网格坐标序列
21        ys.append(round(y, self.digit))
22        y += self.ay
23      for x in xs:                                    # 生成网格字典
24        for y in ys:
25          gridDic[(x, y)] = [(round(zMin, self.digit), 0)]
26      return xs, ys, gridDic                          # 返回X、Y序列和网格字典
```

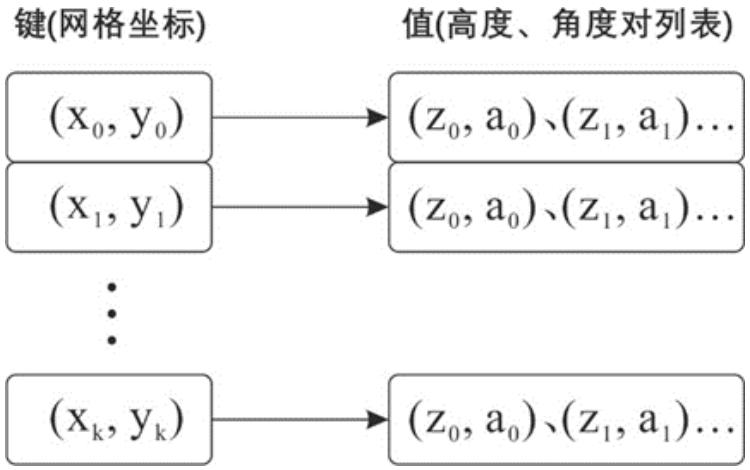
2. 支撑区域识别

- **2.1. 模型投影网格生成：代码分析**
- **FindSptRegion** 类有5个输入参数，包括 stlModel、layers、gridSize、crAngDeg以及xyGap：
  - stlModel　需要生成支撑的STL模型；
  - **Layers**　前续步骤中已经生成的各层切片轮廓数据（参考第五、六、七章）；
  - **gridSize**　网格预设边长，对应式（10-1）或（10-2）中的a；
  - **ax、ay**　用于保存调整后的网格边长；
  - **crAngle**　面片支撑临界角，单位为弧度，“cr”为“critical”缩写，表示临界；
  - **xyGap**　支撑到模型的横向间距，避免打印中在XY平面内支撑材料和模型材料重叠，造成支撑难以去除，默认情况下，该值为1mm。
- **initGrids**函数，首先获取STL模型在X、Y、Z三个方向的极限值，然后根据参数 gridSize 分别计算圆整后的网格边长（这里是变量ax和ay）。函数中定义了几个关键临时变量，包括 xs、ys 以及 **gridDic**，其中 xs 和 ys 为实数列表，分别用来保存X和Y方向的网格坐标序列。
- **initGrids函数返回三部分数据，包括列表 xs、ys 和字典 gridDic。——冗余吗?**

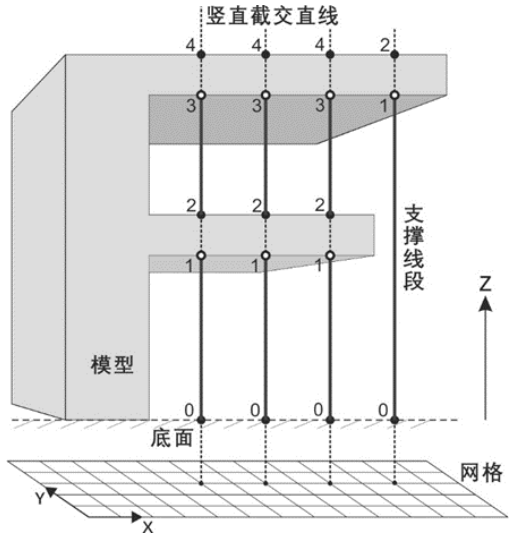
2. 支撑区域识别

• 2.1. 模型投影网格生成：代码分析

- **gridDic**为字典型数据，如左图，键为网格节点坐标（元组），值为一列表变量，**列表元素为过当前网格节点的竖直直线和STL模型的交点信息（元组）**，包括交点的 **Z 坐标值**以及对应面片相对水平面**倾角**（单位为度）。
- 每个列表都预存了一个值为(zMin, 0)的元组元素，这个元素对应右图竖直截交直线和模型底面的交点（即序号为0的交点），元组中0度的倾角表明底面是一个危险面片，设置这个元素的目的单纯是为了保证后续截取支撑线段时的逻辑正确性。gridDic不仅可以表示平面网格信息，还可以表示网格节点上的交点信息，这在后面计算中将十分有用。
- **注意：**在initGrids函数中，在把计算得到的各值存进 xs、ys 及 grids 之前，都会对数值进行**四舍五入操作**，保留指定小数位数，具体小数位数由digit属性确定，默认值为3。目的是统一字典的键，方便后续从gridDic查找数据，避免因数值问题造成查找失败。



字典变量gridDic存储结构



字典变量gridDic存储结构

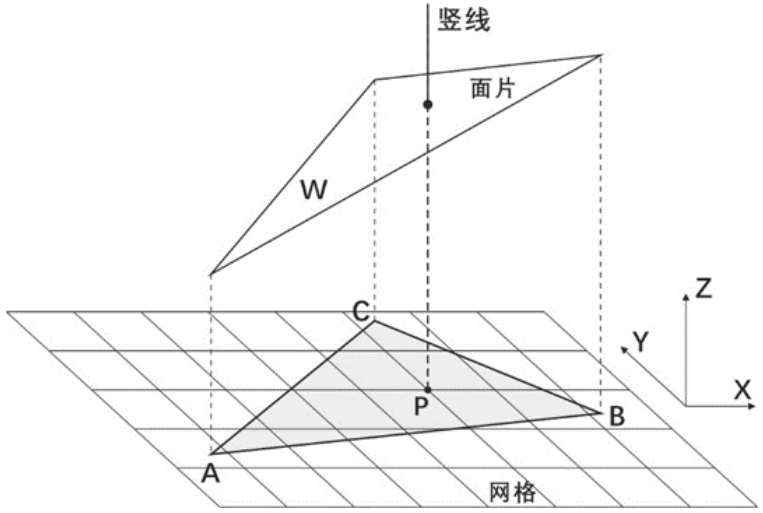
2. 支撑区域识别

• 2.2. 模型支撑点计算

- 对得到的平面网格，在各节点处布置竖线，**计算直线和模型的交点**，并将交点信息存入gridDic字典中，方便后续识别支撑线段。
- **任务：计算：竖线序列和STL模型的交点，即计算：竖线序列和三角面片序列的交点。**

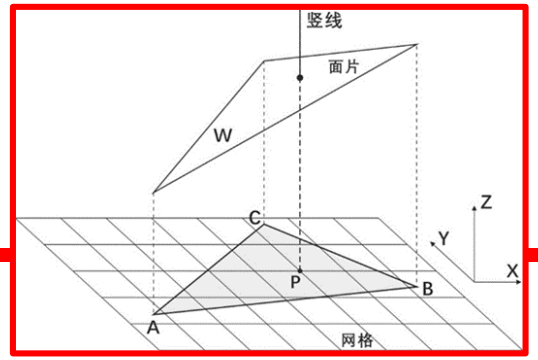
首先需解决：单个竖线 和 单个面片 求交问题——第1种思路

- **直观思路：先求交，后判交**
  - 在第二章中，已实现**直线和平面的求交算法**（GeomAlgo中的intersect函数）。显然，竖线和三角面片的求交问题是直线和平面求交问题的特例。
  - 首先将竖线作为**普通直线**、三角面片作为**普通平面**代入 intersect 函数计算交点，然后判断该交点是否在三角面片内，如果交点在面片内，则竖线和面片相交，此时返回交点，否则竖线和面片不相交。



网格竖线与面片判交及求交计算原理

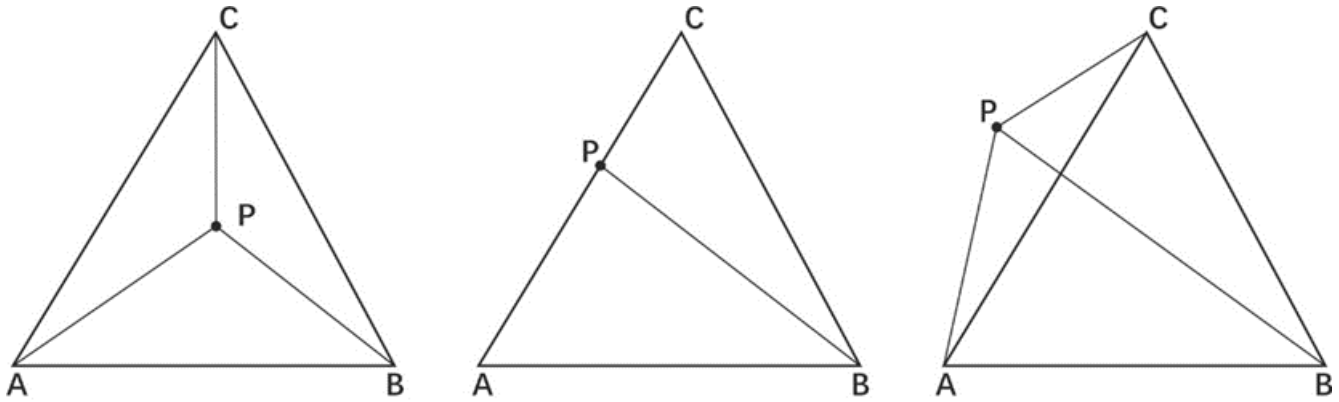
2. 支撑区域识别



• 2.2. 模型支撑点计算

单个竖线 和 单个面片 求交问题——第2种思路

- **逆向思维：先判交，后求交。**
- **判交：**判断空间中任意一条直线和一个三角面片是否相交相对较难！但如果这条直线沿竖直方向，那么判交就变得非常简单。只需在XY平面上检查竖线投影点是否落在三角面片的投影内即可。考虑到竖直线是过网格节点的竖直线，因此只需**判断网格节点是否落在三角面片的投影内**。
  - **实际案例：**记一网格节点为 P(x, y)，空间三角形三个顶点在水平面投影分别为 A、B、C，对应坐标分别为 (x_A, y_A)、(x_B, y_B)、(x_C, y_C)。
  - 要判断点 P 是否落在三角形 ABC 内，可应用 **PointInPolygon** 函数，但相对比较繁琐。
  - 更为便捷方法：当点 P 在三角形 ABC 内部或边上时，小三角形 PAB、PBC、PAC 的面积之和恰好等于三角形ABC面积；而当点P在三角形ABC外部时，三个小三角形的面积之和必然大于三角形ABC面积。



平面上点和三角形的位置关系判断

2. 支撑区域识别

• 2.2. 模型支撑点计算

单个竖线 和 单个面片 求交问题：第2种思路 先判交，后求交

- 下面给出平面上**点和三角形包含关系的判断代码（判交）**：

FindSptRegion.py　寻找支撑区域类：在平面上判断一个点是否在三角形投影内　10-2

```
1   class FindSptRegion:
2       ……                                  # 忽略类中其他代码
3       def pointInTriangle(self, pt, tri, err = 1.0e-7):   # 报告点是否在三角形投影内部
4           A, B, C = tri.A, tri.B, tri.C
5           S = self.area(A, B, C)          # 大三角形面积
6           S1 = self.area(pt, A, B)        # 小三角形面积
7           S2 = self.area(pt, B, C)
8           S3 = self.area(pt, A, C)
9           if math.fabs(S1 + S2 + S3 - S) < err:   # 判断面积是否相等
10              return True                 # 如果相等，则返回True
11          return False                    # 否则返回False
12      def area(self, A, B, C):            # 计算由三点确定的三角形面积
13          x1, x2, x3, y1, y2, y3 = A.x, B.x, C.x, A.y, B.y, C.y
14          return 0.5* math.fabs(x1*y2+x2*y3+x3*y1-x1*y3-x2*y1-x3*y2)
```

- **pointInTriangle** 的作用是在平面上检测一个点是否落在三角形内； **area** 函数用于计算由给定三个顶点围成的三角形面积。

  - **pointInTriangle**函数有3个参数，其中 pt 和 tri 分别为待检测点和三角形，第三个参数err为检测误差，这里给它设置的默认值为1.0×10^{-7}，必须强调的是，**这个参数对函数成败至关重要**。在以上代码中，将 err 参数用在大三角形面积（S）和小三角形面积和（S1 + S2 + S3）的比较上，由于面积计算中数值误差的存在，两者实际上很难恰好相等（即便点在三角形内部时）。
  - **思考：（1）如果将err值设置为0，函数是否还能返回正确结果？（2）进一步地，分析误差err对整个模型支撑点计算的影响（实际上，当err不为0时，计算结果是偏安全的）。**

2. 支撑区域识别

- **2.2. 模型支撑点计算**

单个竖线 和 单个面片求交问题：第2种思路 先判交，后求交

- **求交**：如果判断竖线和面片**相交**，则进一步调用 **GeomAlgo.intersect** 函数求取它们的交点；如果不相交，则返回空。
- 在处理竖线和面片的求交问题时，推荐使用**“先判交、再求交”** 的方法，而不是直接求交。

为什么要这样做（先判交，后求交）？

——有意设计，目的是减少求交计算量。

2. 支撑区域识别

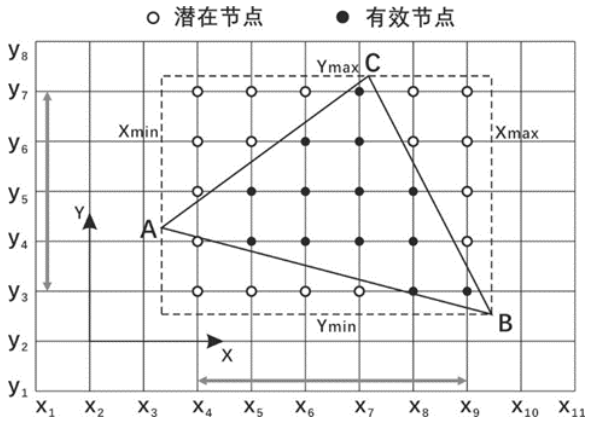
- **2.2. 模型支撑点计算：竖线序列和面片序列求交**
- 如果不计时间成本，可用暴力法进行两两遍历求交，时间复杂度为O(m n)，其中 m 为平面网格节点数，n 为模型中的面片个数。
- 在解决竖线序列和面片序列的求交问题时，一般先入为主的思路是将竖线作为主体去和面片序列求交，但这种思路很难发挥竖线的网格特性，算法优化空间不大。
- 换个角度，将三角面片作为主体去和竖线序列求交，优化空间较大。据此：

① 首先，从面片序列中任意取出一个三角面片（记为W）；

② 对该三角形，遍历平面网格，并根据以上介绍的**面积比较法**找出所有落在三角形W投影内的节点；

③ 然后，根据这些节点构造竖线，求出所有和三角形W相关的交点。

④ 遍历整个面片序列，求出所有交点。

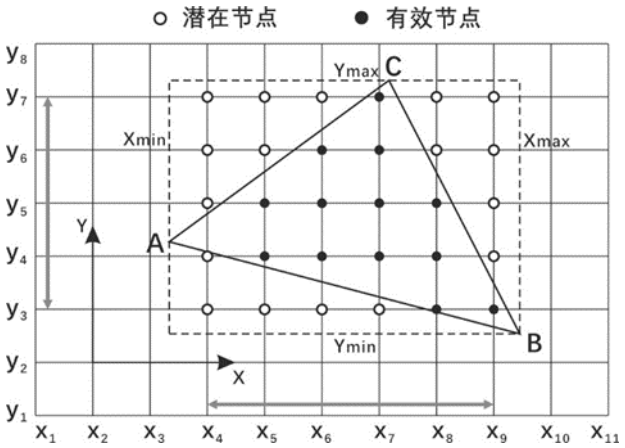


落在面片投影内的有效节点确定方法

- 使用这种方法，针对一个三角形，先通过判交筛选出**有效网格节点**，然后仅对有效节点上的竖线求交，可以**减少求交次数**，提高求交效率。
- 该方法将求交范围限定在了当前三角面片的投影范围内，具体求交次数和网格密度以及面片投影面积大小有关。
- 该方法确实能够降低求交次数，但是**不能降低判交次数**！判交次数仍为 m n。

2. 支撑区域识别

- **2.2. 模型支撑点计算：竖线序列和面片序列求交——优化**
- **潜在节点筛选**：进一步以减少判交次数。—— **原始节点 → 潜在节点 → 有效节点**
  - 如图所示，平面三角形ABC为空间三角面片W在网格平面上的投影，如能事先快速筛选出落在 ABC 内的网格节点，那么就能减少上述判交次数。
  - 但这又回到了如何判断一个点是否落在三角形内的问题，应该避开这个问题，使用更为轻便的方法去做节点筛选。**——先有鸡还是先有蛋的悖论**
  - 如图，**矩形DEFG**为三角形ABC的轴平行**包络矩形**，可通过简单的坐标比较从所有网格节点中筛选出落在矩形DEFG内的**潜在节点**，然后再从这些潜在节点中使用上述判交方法筛选出落在三角形ABC内的有效节点，这样就可以减少判交次数。



落在面片投影内的有效节点确定方法

有效网格节点提取

- 如图，在X方向上，不妨记x_{min}、x_{max}分别为三角形ABC的左右极限，根据 initGrids 函数返回的网格坐标序列 xs，可从中找出落在 $[x_{min}, x_{max}]$ 范围内的子序列（记为 **sub_xs**）；同理，在Y方向上，也可以得到一个子序列 **sub_ys**。
- 然后遍历并两两组合两个子序列中的元素，得到一系列网格节点坐标（潜在节点），并通过面积法检测每个节点坐标是否落在三角形ABC内，如果落在ABC内，则该节点为**有效节点**，否则为无效节点。

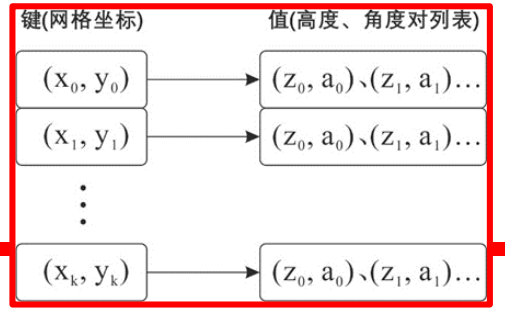
2. 支撑区域识别

- **2.2. 模型支撑点计算：竖线序列和面片序列求交**
- **有效网格节点提取实现代码：**针对STL模型中任意一三角面片，通过 **getValidGrids** 函数筛选出有效节点。该函数包含3个输入参数，其中tri为指定面片，xs、ys分别为网格坐标序列，由initGrids函数生成。函数返回为**有效节点序列**，每个节点为x、y坐标元组。

FindSptRegion.py	寻找支撑区域类：寻找落在指定三角面片投影内的有效网格节点	10-3

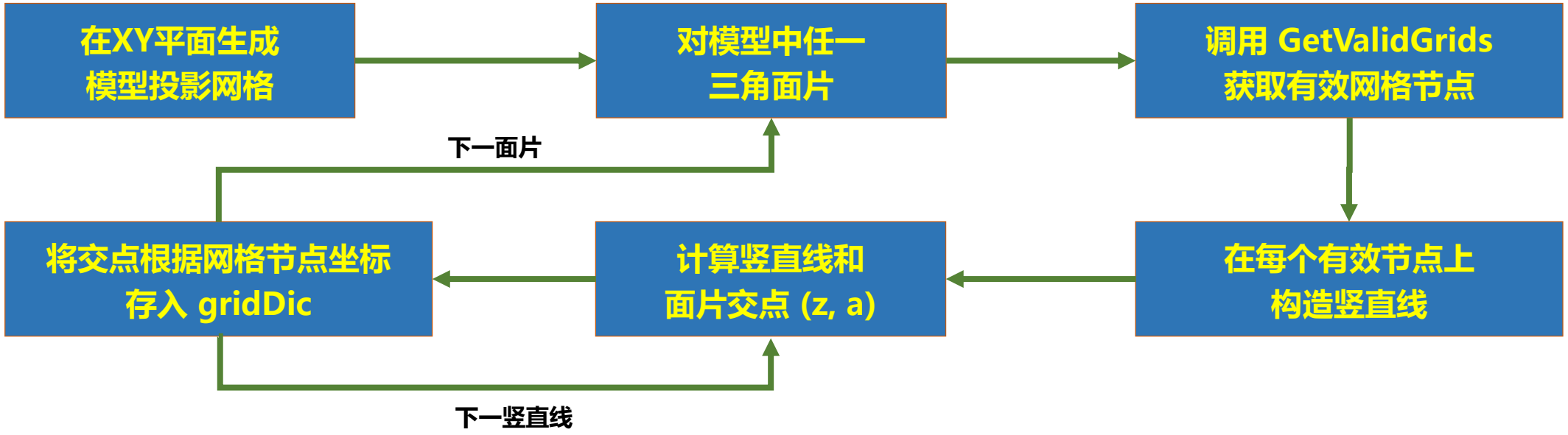
```
1   class FindSptRegion:
2     def getValidGrids(self, tri, xs, ys):                    # 定义寻找有效网格节点函数
3       A, B, C = tri.A, tri.B, tri.C
4       xMin, xMax = min(A.x, B.x, C.x), max(A.x, B.x, C.x)    # 面片在X方向极值
5       yMin, yMax = min(A.y, B.y, C.y), max(A.y, B.y, C.y)    # 面片在Y方向极值
6       sub_xs, sub_ys = [], []                                # 定义子序列
7       for x in xs:                                           # 寻找落在[xMin, xMax]的子序列
8         if x >= xMin and x <= xMax:
9           sub_xs.append(x)
10        elif x > xMax:
11          break
12      for y in ys:                                           # 寻找落在[yMin, yMax]的子序列
13        if y >= yMin and y <= yMax:
14          sub_ys.append(y)
15        elif y > yMax:
16          break
17      grids = []                                             # 用于保存有效节点
18      for x in sub_xs:                                       # 从子序列中筛选落在面片投影内
19        for y in sub_ys:                                     # 的有效网格节点
20          if self.pointInTriangle(Point3D(x, y), tri):       # 判断点在三角形
21            grids.append((x, y))                             # 每个网格节点为(x, y)元组
22      return grids                                           # 返回有效节点序列
```

2. 支撑区域识别

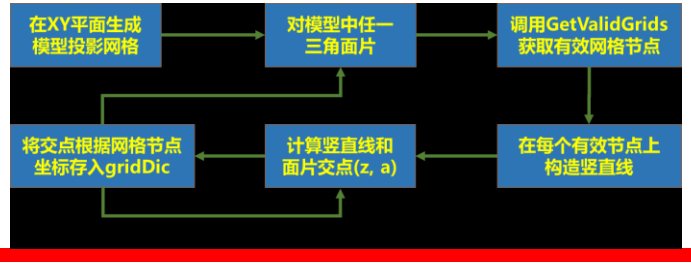


字典变量gridDic存储结构

- **2.2. 模型支撑点计算：竖线序列和面片序列求交**
- **交点保存和计算**
  - **initGrids**函数除返回 xs、ys 序列外，还返回了一个字典对象 gridDic，键为网格节点坐标，值为该节点上方保存交点信息（高度、角度）的序列。将得到的所有交点都**保存**到 gridDic 中
  - 首先根据交点的 x、y 坐标在 gridDic 中**检索**得到相应交点信息序列，再将交点以**元组(z, a)**的形式存入以上序列中，其中 z 为交点 Z 坐标，a 为对应面片倾角。因此，在以上竖线和三角面片的交点计算过程中，需要记录面片倾角信息。
- **综上，竖线序列和面片序列求交（模型支撑点计算）步骤如下：**



2. 支撑区域识别



- **2.2. 模型支撑点计算：竖线序列和面片序列求交**
- 以上竖线序列和面片序列求交方法的时间复杂度接近O(n)，其中n为STL模型中面片个数，相较于暴力法，该方法能大幅提高线面序列求交效率，这主要得益于对竖线网格特性的巧妙应用。最后给出针对给定STL模型的支撑点计算函数 **calcModelSptPoints:**

FindSptRegion.py | 寻找支撑区域类：STL模型支撑点计算 | 10-4

```
1   class FindSptRegion:
2       def calcModelSptPoints(self):                  # 定义计算模型支撑点函数
3           xs, ys, gridDic = self.initGrids()         # 初始化网格，返回网格字典
4           for tri in self.stlModel.triangles:        # 遍历STL模型中每个三角面片
5               validGrids = self.getValidGrids(tri, xs, ys)  # 获取当前面片有效网格节点
6               for vg in validGrids:                  # 遍历每个有效节点
7                   ln = Line(Point3D(vg[0], vg[1]), Vector3D(0, 0, 1)) # 构造过节点竖线
8                   pln = Plane(tri.A, tri.N)          # 构造三角面片所在平面
9                   pt = intersect(ln, pln)            # 线面求交，获得交点pt
10                  if pt is not None:                 # 如果pt不为空
11                      z = round(pt.z, self.digit)    # 获取交点z坐标
12                      a = self.getFacetAngle(tri)    # 获取面片倾角a
13                      gridDic[vg].append((z, a))     # 构造(z, a)，并将其添加至字典
14          for k in gridDic:                          # 遍历字典
15              gridDic[k].sort(key = lambda t : t[0]) # 对每组交点按z值从小到大排序
16          return gridDic                             # 返回网格节点
17      def getFacetAngle(self, tri):                  # 定义计算面片倾角函数
18          angle = tri.N.getAngle(Vector3D(0, 0, 1))  # 角度单位为度
19          angle = round(angle, self.digit)           # 角度保留指定小数位数
20          return (math.pi - angle) if angle > math.pi/2 else angle        # 保证倾角在0~90度范围内
```

倾角设定在0~90度范围内

强调：

- 函数最后进行了一项排序操作，即对每条竖线上的交点按其Z坐标从小到大排序，其作用是方便后续根据交点的奇偶属性提取支撑线段。
- 函数返回为gridDic字典。

2. 支撑区域识别

- **2.3. 切片支撑点计算**

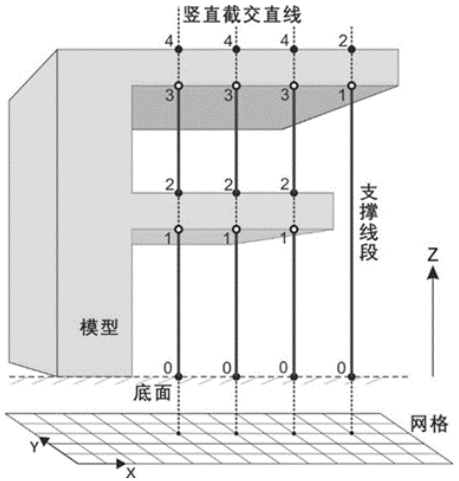
使用以上实现的 calcModelSptPoints 函数，可计算网格竖线和三维模型之间的交点。只要校验交点序号的奇偶性，就能提取到用于支撑模型悬臂结构的**支撑线段**。

问题提出：如何将支撑线段转化为实际可用的支撑和支撑路径?

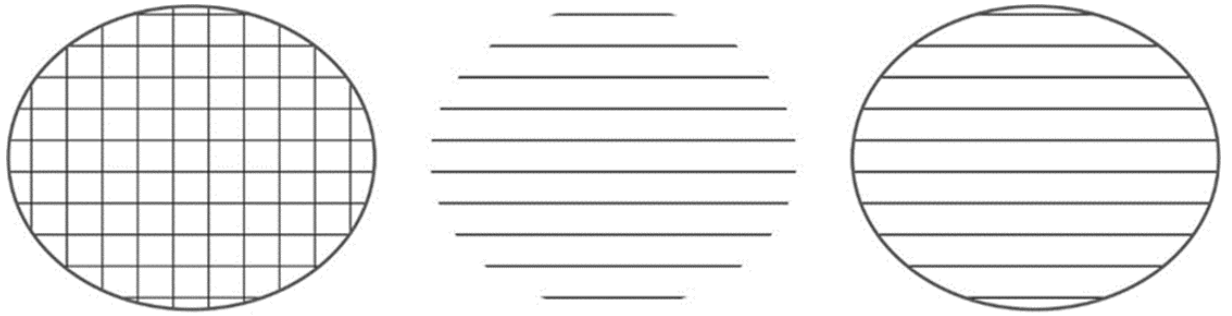
一种思路：让打印材料在竖直方向上直接沿着支撑线段生长，以得到一系列细长的支撑柱子。

——考虑到这些细长柱子在打印过程中极易**倒塌**，因此这种思路**意义不大**。

难点：支撑线段是沿 Z 轴的一系列线段，而支撑路径应和模型切片一样，位于 XY 平面上，它们之间如何转化?



支撑线段



XY平面上的支撑路径

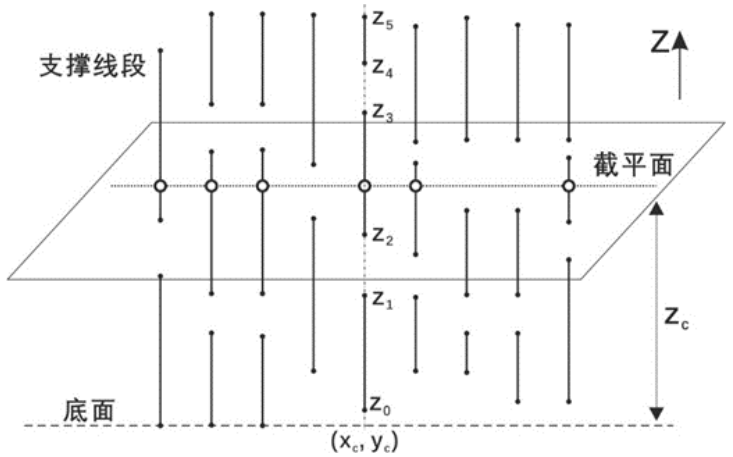
2. 支撑区域识别

- **2.3. 切片支撑点计算**

竖直方向支撑线段 → 水平方向支撑路径

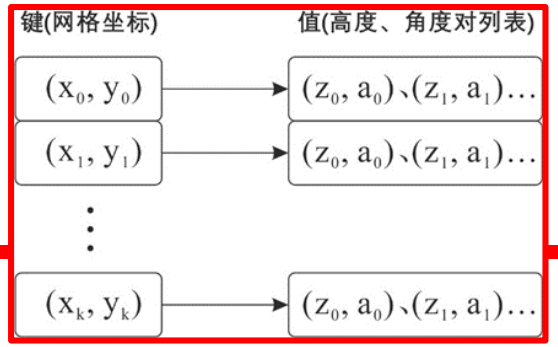
无论如何，先切片！

- **尝试（任务）：** 对得到的支撑线段进行切片，即计算**指定高度**的**水平面**和**支撑线段**的交点。区别于模型支撑点（即模型和网格竖线的交点），称这些交点为**切片支撑点**。



支撑线段与截平面相交计算

2. 支撑区域识别



- **2.3. 切片支撑点计算**

- **已知条件：** **calcModelSptPoints** 函数返回字典变量 **gridDic**，其中字典的键为网格节点坐标，值为该节点正上方的一组交点，且交点按其 Z 坐标从低到高进行排序。

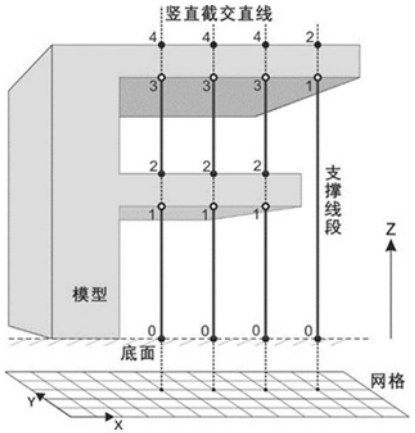
以 gridDic 中其中一个网格节点上对应的一组交点为例说明支撑线段的提取及切片方法（即切片支撑点计算方法）

不妨记这个网格节点坐标为 (x_c, y_c)，对应的交点序列记为 ξ，

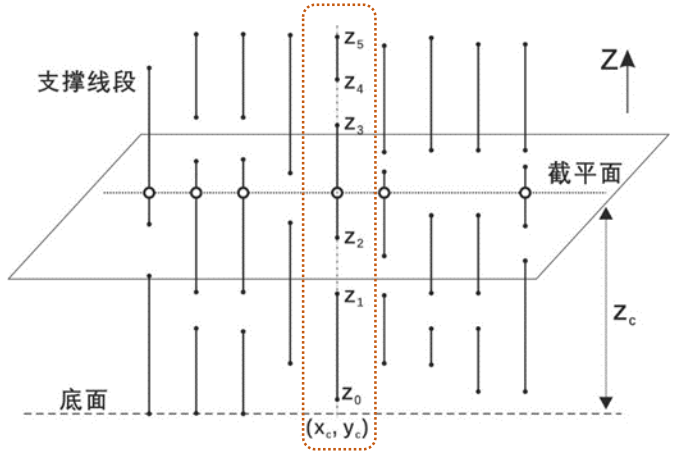
$$\xi = \{(z_i, a_i) \mid i \in [0, 2n]\},$$

其中 z_i 为交点 Z 坐标，a_i 为对应面片水平倾角，n为自然数，且如果 $0 \le i < j \le n$，则 $z_i < z_j$。

正常情况下，每个节点上方总会有**奇数（2n + 1）个交点**。这是合乎逻辑的，因为经过节点的竖线总是从三维模型的外部刺透模型，这样必然会产生偶数个交点，再加上在模型底面人为设置的一个交点，因此总共会有奇数个交点。



支撑线段



支撑线段与截平面相交计算

2. 支撑区域识别

• 2.3. 切片支撑点计算

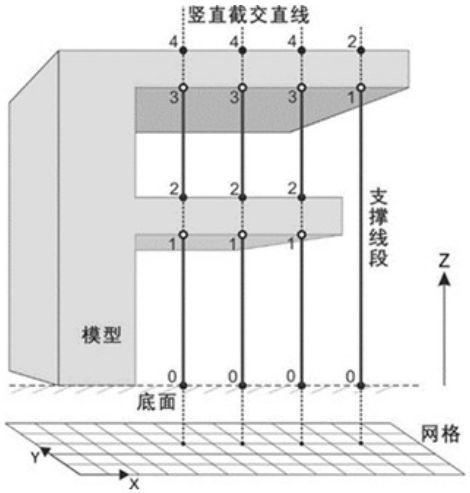
- 一条潜在支撑线段的**下支点**只可能取在 (z_{2k}, a_{2k}) 上，其中 $k \in [0, n-1]$，相应**上支点**为 (z_{2k+1}, a_{2k+1})。如果该线段为支撑线段，上支点还必须满足 $a_{2k+1} \le \theta_0$，其中 θ_0 为面片支撑临界角，任何倾角小于临界角的面片都是危险面片。

对一张高度为 z_c 的截平面，要找出该平面和支撑线段的交点，只需逐对遍历 ξ 中相邻交点，并判断

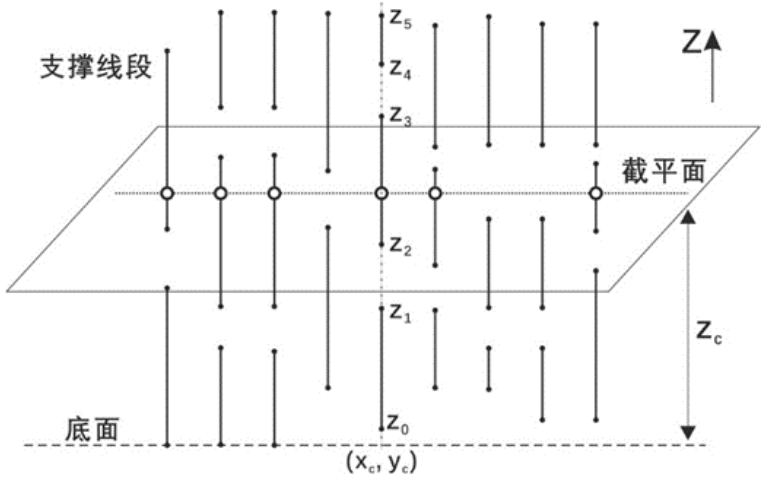
$$z_{2k} \le z_c \le z_{2k+1} \qquad (10\text{-}3)$$

$$a_{2k+1} \le \theta_0 \qquad (10\text{-}4)$$

两式是否同时成立。如果成立，则截平面和当前网格节点上的支撑线段的交点坐标为 (x_c, y_c, z_c)。如果遍历 ξ 后以上条件均无法同时成立，则截平面和当前网格节点上的支撑线段无交点。



支撑线段



支撑线段与截平面相交计算

2. 支撑区域识别

• 2.3. 切片支撑点计算

- 以上解决了单个网格节点上方支撑线段和截平面的求交问题，同理，只要**遍历网格字典 gridDic 中的所有节点，就能找出和当前高度截平面相交的所有切片支撑点。**
- 下面给出切片支撑点的计算函数，函数名为 **calcLayerSptPoints**，这部分代码写在 FindSptRegion 类中，具体实现如下：

FindSptRegion.py	寻找支撑区域类：切片支撑点计算	10-5

```
1   class FindSptRegion:
2   def calcLayerSptPoints(self, gridDic, z):      # 定义计算切片支撑点函数
3       pts = []                                   # 定义切片支撑点序列
4       for key in gridDic.keys():                 # 遍历网格字典
5           zas = gridDic[key]                     # 获取指定网格上的交点序列
6           if self.hasSptPoint(zas, z):           # 如果网格节点上有切片支撑点
7               pts.append(Point3D(key[0], key[1], z))        # 则构造支撑点并存储在pts中
8       return pts                                 # 返回切片支撑点序列
9     def hasSptPoint(self, zas, z):               # 判断网格上是否有切片支撑点
10      for i in range(0, len(zas), 2):            # 逐对遍历序列中相邻交点
11          if i + 1 <= len(zas) - 1:              # 保证数组索引不越界
12              za0, za1 = zas[i], zas[i+1]
13              if za1[1] < self.crAngle and z >= za0[0] and z<= za1[0]:      # 如果满足式
14                  return True                    # (10-3)、(10-4)条件，则返回True
15      return False                               # 否则返回False，无切片支撑点
```

是判交
而非求交问题
坐标点都是已知的

2. 支撑区域识别

- **2.3. 切片支撑点计算**

- 以上代码实现了两个函数， **hasSptPoint** 函数和 **calcLayerSptPoint** 函数。
- 其中 **hasSptPoint** 函数为 calcLayerSptPoint 函数的子函数，它的**输入参数为 zas 和 z**，其中 zas 为某个指定网格节点上的交点序列（即前面提到的 ξ），z 为截平面高度，这个函数的作用是根据式(10-3)、(10-4)判断网格节点上方是否存在切片支撑点，其**返回值为布尔类型**。
- 相对来说，**calcLayerSptPoint** 函数的实现比较简单，它的**输入为网格字典 gridDic 和切片高度 z**，通过遍历gridDic及调用hasSptPoint函数，**输出切片支撑点序列**。有了这个函数，只要指定不同的切片高度，就能得到不同高度上的切片支撑点。

✓ **至此，在 FindSptRegion 类中定义并实现了若干函数。**
✓ **通过这些函数铺垫，首先，在竖直方向上，计算得到了一系列模型支撑点，基于此，在水平面上，进一步计算得到了指定高度上的切片支撑点。**
✓ **接下来需要将焦点放在这些切片支撑点上，着重讲解如何将这些切片支撑点转化为平面支撑区域。**

2. 支撑区域识别

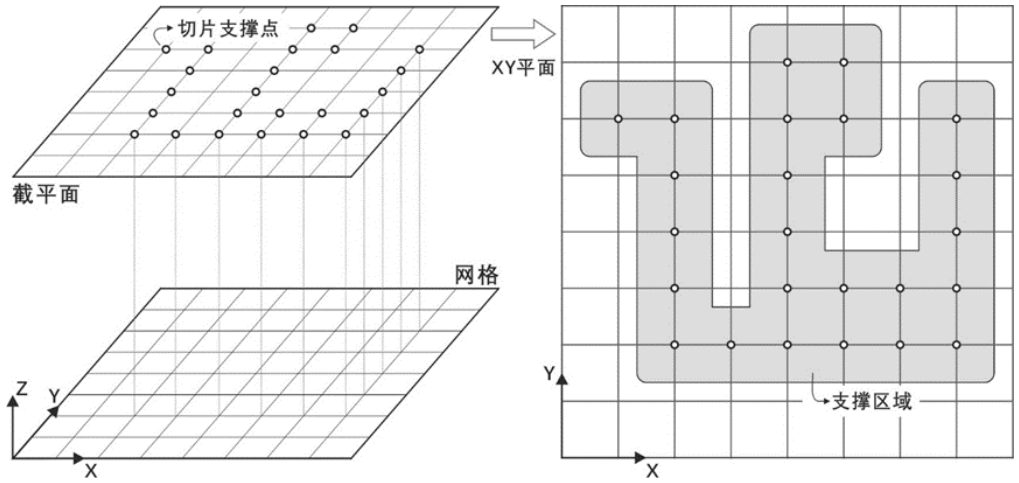
• 2.4. 切片支撑区域生成

- 对得到的某一截平面上的切片支撑点（散乱分布在平面上，竖直投影落在网格节点上），可以想象，网格尺寸越小，即网格节点越密，在平面上得到的切片支撑点就越多越密。

理论上

网格尺寸趋向无穷小 → 无穷多切片支撑点 —密密麻麻→ 在当前切平面上呈现支撑区域的模样

- 实际上，由于计算资源限制，不可能将网格尺寸设置得非常小，最终得到的可能是**稀疏**分布在平面上的**有限个**切片支撑点。



切片支撑点转化为平面支撑区域示意

任务

寻找一种通用方法和策略，根据截平面上的**稀疏支撑点**生成**支撑区域**。

2. 支撑区域识别

• 2.4. 切片支撑区域生成

截平面上

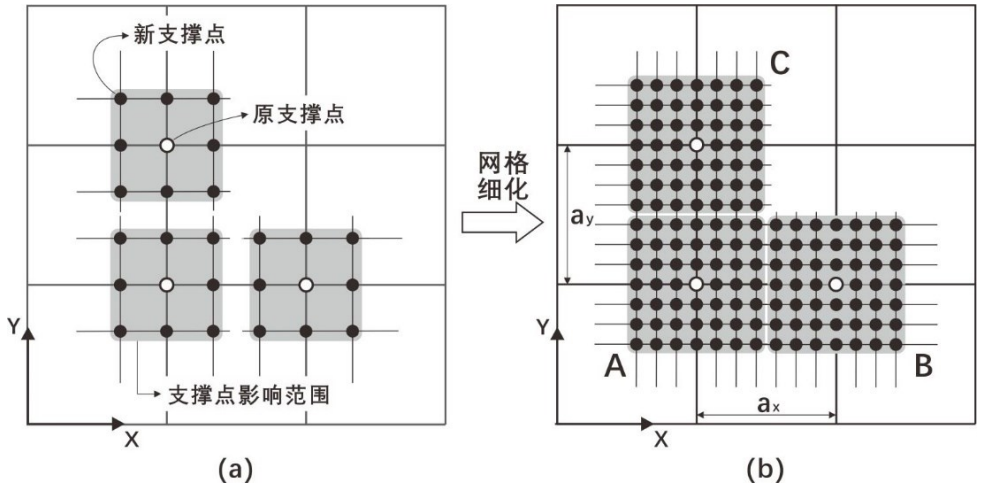
一个思想实验

当网格间距从初始值逐渐变化到0的过程中，必然会有新的切片支撑点（虚拟支撑点）出现在原有支撑点附近

随着网格间距进一步减小，网格**分辨率进一步提高**，支撑点附近出现新支撑点的区域范围将逐渐明确

并且，对两相邻原支撑点，它们相应的出现新支撑点的范围可能**逐渐融合**，并在最后**合并为一个区域**

根据式(10-1)、(10-2)，平面网格间距由a_x、a_y控制，
可以想象在原支撑点附近做若干竖直直线，试探这些直线是否和三维模型上的危险面片相交



网格细化过程中在原切片支撑点附近出现的新支撑点

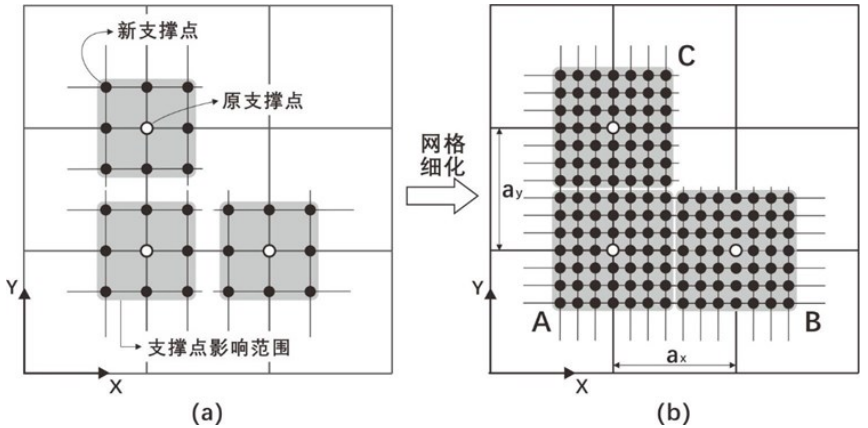
注意：

严格讲，这个思想实验不一定完全正确，
但却能在一定程度上能够启发寻找
将平面支撑点转化为支撑区域的方法。

2. 支撑区域识别

• 2.4. 切片支撑区域生成

- 根据以上思想实验，可定义**支撑点的影响范围**概念，即在该支撑点附近可能存在新支撑点的最大范围。简单起见，不妨假设这个**影响范围是一个矩形**。
- **矩形的尺寸确定方法**
  - 如图，网格节点 A、B、C 上各有一个支撑点，在 X 方向上，当 a_x 不断减小时，节点 A、B 上的支撑点影响范围逐渐清晰并融合。
  - 临界情况下，当两个矩形在 X 方向的长度尺寸均为 a_x 时，相应左右边界完全融合，此时再增加矩形的 X 尺寸已经不能影响它们在边界的融合效果，因此在 X 方向上，可认定矩形的长度尺寸为 a_x。
  - 同理，在 Y 方向上，通过分析节点 A、C 上的支撑点影响范围，可认定矩形的高度尺寸为 a_y。
- 根据以上假设及分析，**一个支撑点的影响范围为：中心在该点的轴平行矩形，且矩形的在 X、Y 方向上的边长分别为 a_x、a_y。**



网格细化过程中在原切片支撑点附近出现的新支撑点

2. 支撑区域识别

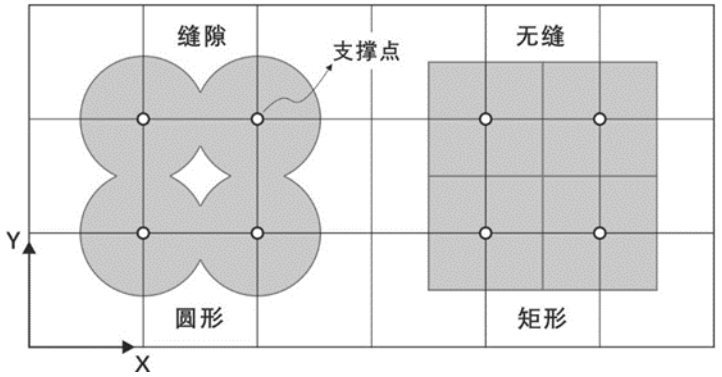
- **2.4. 切片支撑区域生成**
- **使用矩形假设的原因**

首先，矩形设定是一种偏安全的设定。

•对三维模型支撑生成来说，如果在需要添加支撑的地方没有添加支撑可能导致打印过程中模型坍塌，但如果**在某些不需要添加支撑的地方添加了冗余支撑，则并不会对打印过程带来不良影响。**因此，对支撑点的影响范围设定来说，即便矩形内某些新支撑点对应的三角面片是安全面片，也能接受。

其次，矩形设定更易于相邻区域融合。

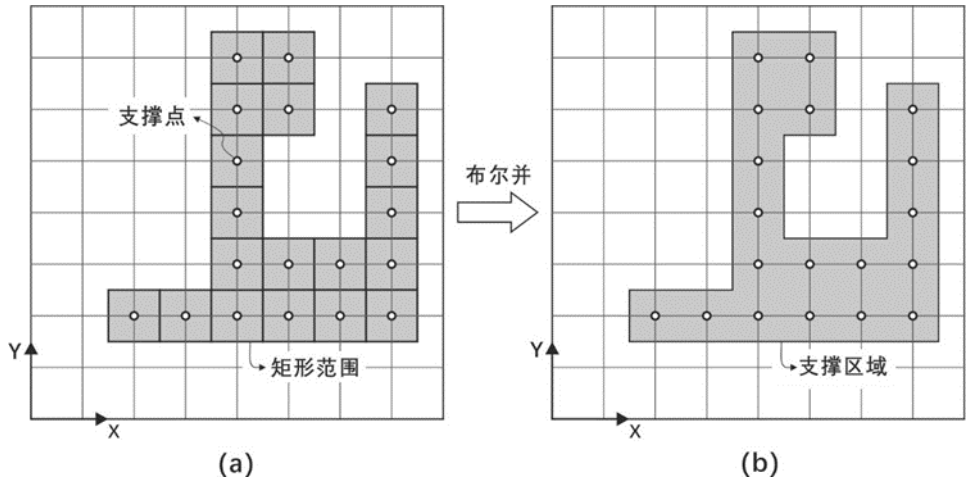
•如图所示，不失一般性，其实也可以支撑点的影响范围设定为圆形。但是，在对相邻支撑点的影响范围进行融合时，两个圆形之间容易留下缝隙，为了覆盖这些间隙，不得不将圆的直径设置得很大，但这样会带来更多不必要的冗余支撑。而如果使用矩形设定，能够以较小的尺寸实现相邻支撑点影响范围的无缝融合。



切片支撑点影响范围圆形和矩形设定优劣对比

2. 支撑区域识别

- **2.4. 切片支撑区域生成**
- **矩形融合**
- 对平面上的一系列支撑点，已生成各个支撑点的影响范围矩形，每个矩形的长宽尺寸分别为 a_x 和 a_y。**接下来需要对各个孤立的矩形区域进行“融合”操作，生成一个（或多个）支撑区域。**
- **如何融合矩形区域?**
  - 布尔运算！事实上，矩形区域的融合问题就是求这些区域的布尔并运算。
  - 关于布尔运算，第八章已经介绍了一个非常强大的库——**Clipper**，而且，还构建了Clipper库的适配器类**ClipperAdaptor**，方便交换不同的多边形数据。
  - 因此，只要将上述各个孤立矩形输入Clipper库中的布尔并运算函数，就能输出所要求的支撑区域。



切片支撑点生成平面支撑区域原理

布尔并
Union

2. 支撑区域识别

- **2.4. 切片支撑区域生成： genRawSptRegion实现**
- 下面在FindSptRegion类中添加支撑点到支撑区域的转化函数，函数名为**genRawSptRegion**，具体实现如下：

FindSptRegion.py	寻找支撑区域类：在截平面上将切片支撑点转化为初始支撑区域	10-6

```
1   ……                                          # 导入相关库
2   class FindSptRegion:
3     ……                                        # 忽略类中其他代码
4     def genRawSptRegions(self, pts):          # 支撑点转化为初始支撑区域函数
5       rects = []                              # 支撑点影响范围矩形序列
6       for pt in pts:                          # 逐个将支撑点转化为矩形
7         rects.append(self.pointToRect(pt, 1.1*self.gridSize, 1.1*self.gridSize))
8       clipper, ca = Pyclipper(), ClipperAdaptor()   # 裁剪对象clipper，适配器对象ca
9       clipper.AddPaths(ca.toPaths(rects), PT_SUBJECT)      # 逐个将矩形添加至clipper
10      sln = clipper.Execute(CT_UNION, PFT_POSITIVE)        # 执行布尔并运算
11      return ca.toPolys(sln, pts[0].z)          # 返回布尔并运算后区域
12    def pointToRect(self, pt, lx, ly):          # 定义将点转化为平面矩形函数
13      vx, vy = Vector3D(lx/2, 0), Vector3D(0, ly/2)   # 矩形沿X、Y方向半边向量
14      rect = Polyline()                         # 定义一个矩形，实为多边形
15      rect.addPoint(pt - vx - vy)               # 根据矩形中心点和半边向量
16      rect.addPoint(pt + vx - vy)               # 逐个添加矩形各个顶点
17      rect.addPoint(pt + vx + vy)
18      rect.addPoint(pt - vx + vy)
19      rect.addPoint(pt - vx - vy)
20      return rect                               # 返回矩形
```

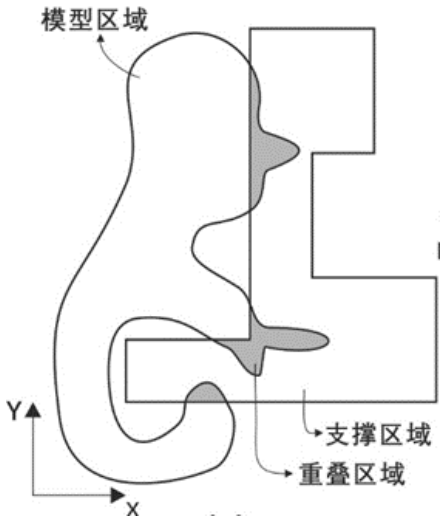
为什么
1.1倍
网格尺寸

2.支撑区域识别

- **2.4. 切片支撑区域生成： genRawSptRegion分析**
- 以上代码定义并实现了两个函数，**genRawSptRegions** 和 **pointToRect**。
- **pointToRect** 是子函数，它的作用是在 XY 平面上以给定点为中心生成一个轴平行矩形，在它的三个输入参数中，pt 为给定点，lx 和ly 分别用于指定矩形在 X 和 Y 方向上的边长。为了配合 Clipper 库的数据结构，在 pointToRect 函数中，将矩形定义为一个拥有5个顶点且首尾点重合的多段线（Polyline），各个顶点呈逆时针分布。
- **genRawSptRegions**
  - **输入**：pts，即切片支撑点序列，该数据具体可通过调用calcLayerSptPoints函数获取；
  - **输出**：当前截平面上的（初始）支撑区域。
  - **原理**：借助Clipper库，该函数的实现过程非常简单！
    - 首先根据输入的pts生成平面矩形序列rects，然后通过Clipper库对rects序列进行布尔并运算，最终返回表示支撑区域的平面多边形序列。在这过程中，使用了ClipperAdaptor类来转化多边形数据。
    - 需要说明的是：在调用pointToRect函数时，并没有按照上文推导的数据（即ax、ay）去设置矩形边长，而是对lx和ly参数同时**设置了1.1倍的gridSize值**。这是因为a_x和a_y本身根据gridSize值圆整而来，且有$a_x \leq gridSize$, $a_y \leq gridSize$，使用较大的矩形来替换较小矩形可以增加相邻矩形间的重叠面积，**降低Clipper库在布尔并计算中出错的可能性**。

2. 支撑区域识别

- **2.4. 切片支撑区域生成**
- **genRawSptRegions —— 为什么是Raw?**
- 表明这个函数仍然是一个中间函数，它得到的平面支撑区域仍然是初始数据，需要进一步处理，而不是最终结果。
  - 如图所示，因为使用了支撑点影响范围的矩形假设，将每个支撑点的影响范围都等效成大小一致的矩形，导致最终融合得到的支撑区域**侵占**了模型区域，即导致切平面上模型轮廓和支撑轮廓相交。
  - 如果允许这种情况发生，打印时支撑材料将和模型材料混叠，增加打印后支撑从模型上**剥离**的难度。因此，应该在生成支撑区域时避免这种情况发生。



模型区域和支撑区域相交重叠

2. 支撑区域识别

- **2.4. 切片支撑区域生成**
- **模型轮廓和支撑轮廓重叠区域消除**

模型轮廓来自对三维模型的切片，是**精确**且真实存在的轮廓，不容更改，否则打印得到的工件将无法匹配设计模型，从而失去打印本身的意义

支撑轮廓是根据切片支撑点生成的**虚拟**轮廓，它的存在依赖于我们做出的一些假设，如支撑点影响范围的矩形假设，显然这些假设是不具备几何精度的

策略：

对支撑区域轮廓进行修正，使其避开模型轮廓

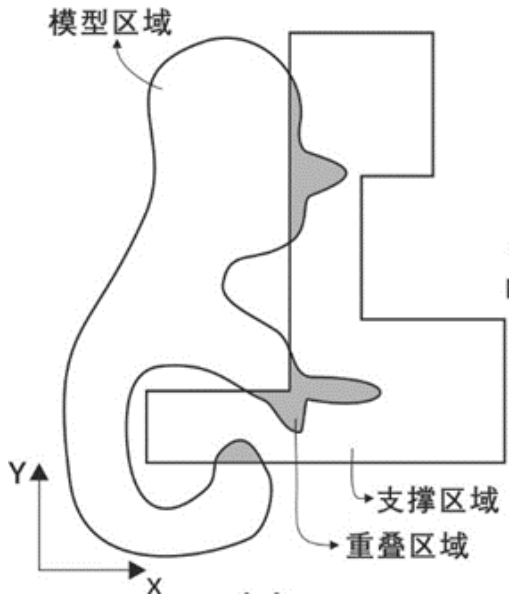
- **如何修正支撑轮廓呢？——布尔运算！**

重叠区域 = 支撑区域 ∩ 模型区域

目标区域 = 支撑区域 — 重叠区域

目标区域 = 支撑区域 — 模型区域

新支撑区域的部分边界恰好和模型区域的部分边界重合，但两区域已**不再相交**



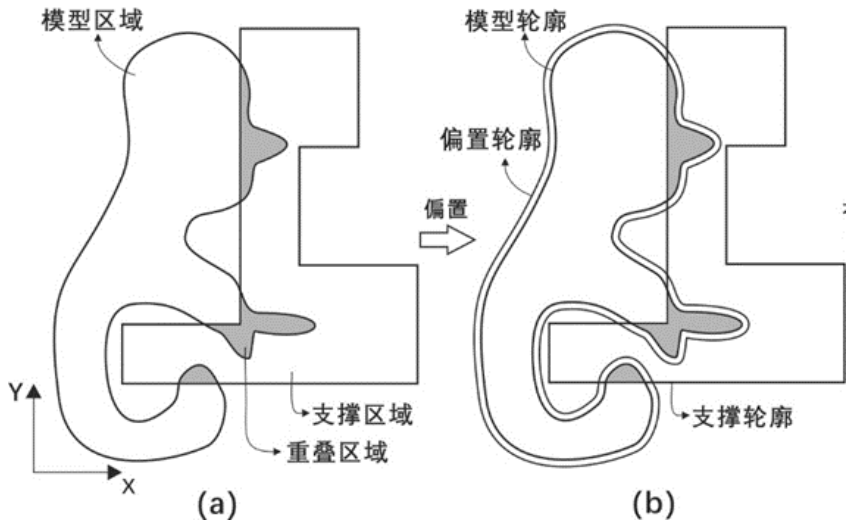
模型区域和支撑区域相交重叠

2. 支撑区域识别

- **2.4. 切片支撑区域生成**
- **模型轮廓和支撑轮廓重叠区域消除**

考虑到打印工艺需求，避免支撑材料和模型材料**过度粘连**，方便打印完成后支撑剥离，应该在模型轮廓和修正后的新支撑轮廓之间**预留一定的间隙**。因为这个间隙是XY平面上轮廓间的间隙，因此称其为 **xyGap（见FindSptRegion类）**

——显然，这个间隙可用**偏置**方法实现。



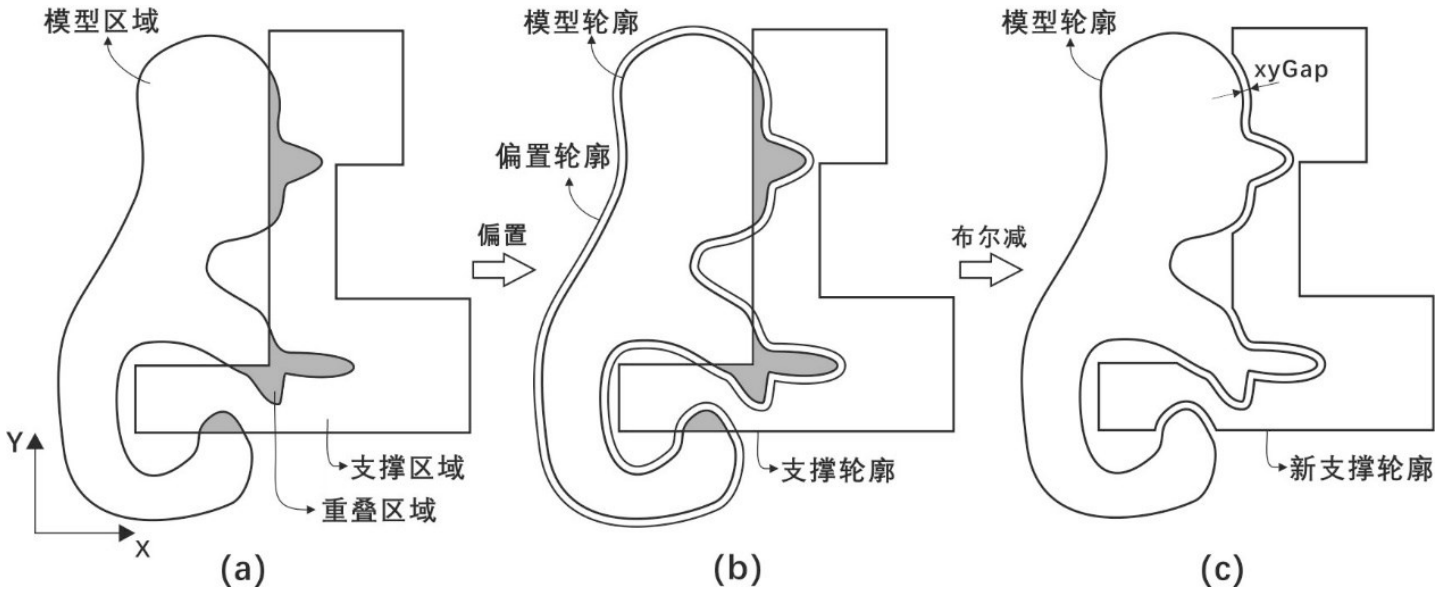
考虑模型区域和支撑区域重叠情况下的支撑区域修正流程示意

2. 支撑区域识别

- **2.4. 切片支撑区域生成**
- **模型轮廓和支撑轮廓重叠区域消除**

根据以上分析，支撑轮廓的修正流程如下：

- 对模型轮廓向外偏置，偏置距离为 xyGap，得到一条（或多条）偏置轮廓，如图b所示；
- 对支撑轮廓和以上得到的偏置轮廓执行布尔减操作，生成的差轮廓即为所求的新支撑轮廓，如图c所示。



新支撑区域 =
支撑区域 — 模型区域的偏置

考虑模型区域和支撑区域重叠情况下的支撑区域修正流程示意

2. 支撑区域识别

• 2.4. 切片支撑区域生成

- 下面给出 **genRawSptRegions** 函数的升级版本——**genSptRegions**，去掉了“raw”字段，它的具体实现如下：

FindSptRegion.py　寻找支撑区域类：实现初始支撑区域修正函数，实现类的核心函数　10-7

```
1   ……                                          # 导入相关库
2   class FindSptRegion:
3     ……                                        # 忽略类中其他代码
4     def genSptRegions(self, pts, layer):      # 定义支撑区域生成函数
5       rawRegions = self.genRawSptRegions(pts) # 根据支撑点生成初始支撑区域
6       clipper, ca = Pyclipper(), ClipperAdaptor()  # 定义裁剪及适配器对象
7       contours = ca.offset(layer.contours, self.xyGap)      # 生成模型轮廓的偏置轮廓
8       clipper.AddPaths(ca.toPaths(rawRegions), PT_SUBJECT)    # 裁剪对象，初始支撑区域
9       clipper.AddPaths(ca.toPaths(contours), PT_CLIP) # 裁剪对象，偏置轮廓
10      sln = clipper.Execute(CT_DIFFERENCE)        # 执行布尔减操作
11      return ca.toPolys(sln, layer.z)             # 返回修正后的支撑区域
```

- **genSptRegions** 和 **genRawSptRegions** 两个函数都有一个名为pts的参数，表示切片支撑点，但前者比后者多了一个名为layer的参数，这个参数表示三维模型在某一层上的切片轮廓数据。根据之前的分析，layer参数的作用是显而易见的，即用于修正genRawSptRegions函数得到初始支撑区域。

- 和 **genRawSptRegions** 一样，**genSptRegions** 函数的实现同样非常简单，主体都是布尔运算。首先调用了genRawSptRegions函数生成了截平面上的初始支撑区域，然后对layer中的模型轮廓做偏置得到偏置轮廓，最后使用Clipper库对两者进行布尔减操作，得到的差轮廓所包围的区域就是当前截平面上所求的支撑区域。

2. 支撑区域识别

• 2.4. 切片支撑区域生成

- FindSptRegion类的**excute**函数实现：

FindSptRegion.py　寻找支撑区域类：实现初始支撑区域修正函数，实现类的核心函数　10-7

```
1   ……                                                  # 导入相关库
2   class FindSptRegion:
3     ……                                                # 忽略类中其他代码
4   def execute(self):                                  # 实现类的核心流程控制函数
5       gridDic = self.calcModelSptPoints()             # 生成模型支撑点
6       for layer in self.layers:                       # 遍历模型切片数据
7         layer.sptContours = []                        # 在每个切片上定义支撑轮廓序列
8         pts = self.calcLayerSptPoints(gridDic, layer.z)  # 计算切片支撑点
9         if len(pts) > 0:                              # 如果支撑点数不为0，
10          layer.sptContours = self.genSptRegions(pts, layer)   # 则生成支撑轮廓
11  def findSptRegion(stlModel, layers, gridSize, crAngle, xyGap = 1):          # 定义支撑生成全局函数
12    FindSptRegion(stlModel, layers, gridSize, crAngle, xyGap).execute()
```

输入参数包括
stlModel（STL模型）
ayers（切片数据）
gridSize（网格尺寸）
crAngle（危险面片临界角度）
xyGap（支撑到模型的横向间距），**输出为切片上的支撑轮廓**

- 首先调用**calModelSptPoints**生成了模型支撑点，然后遍历类中保存的切片序列layers；
- 对切片（layer），首先调用**calcLayerSptPoints**函数生成切片上的支撑点pts，最后将pts和layer输入以上介绍的**genSptRegions**函数生成当前切片上的支撑轮廓。
- **说明：**在execute函数中，动态为每个layer对象添加了一个**sptContours**属性，顾名思义，这个属性的作用是保存计算得到的切片支撑轮廓序列。
- **强调：**在调用calcLayerSptPoints函数生成切片支撑点时，为其指定的截平面高度（即函数的第二个参数）为当前layer的高度，这样做的目的当然是为了保证支撑轮廓在当前切片上。
- 全局接口函数**findSptRegion：**输入参数和FindSptRegion类的初始化函数完全一样，包括stlModel（STL模型）、layers（切片数据）、gridSize（网格尺寸）、crAngle（危险面片临界角度）、xyGap（支撑到模型的横向间距），输出为切片上的支撑轮廓。

2. 支撑区域识别

• 2.5. 支撑区域生成测试

- FindSptRegion 类的测试代码，直接写在FindSptRegion模块的main函数中：

FindSptRegion.py	main函数，对FindSptRegion类进行测试	10-8

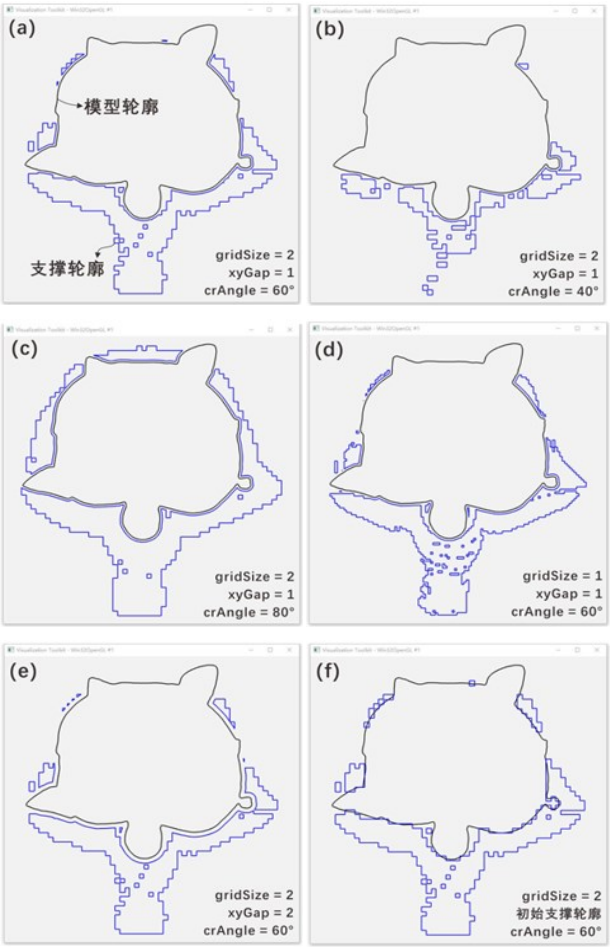
```
1   ……                                          # 导入相关库，忽略模块中其他代码
2   if __name__ == '__main__':                  # 定义main函数
3       src, stlModel = vtk.vtkSTLReader(), StlModel()
4       src.SetFileName("E:\\Models\\STL\\ monk.stl")   # 读取monk.stl文件
5       stlModel.extractFromVtkStlReader(src)           # 提取VTK数据源
6       layers = readSlcFile("D:\\monk at 2.0mm.slc")   # 读取相应切片文件
7       findSptRegion(stlModel, layers, 2, degToRad(60), 1)   # 寻找支撑区域
8       va = VtkAdaptor()                               # 调用VTK显示计算结果
9       # va.drawPdSrc(src).GetProperty().SetOpacity(0.5)   # 按需显示STL模型
10      for i, layer in enumerate(layers):              # 遍历得到的切片序列
11          # if i != 22: continue                      # 按需只显示指定层上轮廓
12          for poly in layer.contours:                 # 显示当前层模型轮廓
13              va.drawPolyline(poly).GetProperty().SetColor(0, 0, 0) # 轮廓显示为黑色
14          for poly in layer.sptContours:              # 显示当前层支撑轮廓
15              va.drawPolyline(poly).GetProperty().SetColor(0, 0, 1) # 轮廓显示为蓝色
16      va.display()
```

- 选用monk模型，包含约33万个三角面片，Z向高度约为176mm。针对monk模型，第八章已经生成了的不同切片厚度（分别为0.2mm、0.5mm、1.0mm、2.0mm）的4个SLC文件，这里选用切片厚度为2.0mm的SLC文件作为模型切片轮廓的来源。
- 对findSptRegion函数的输入参数，除了保持模型（stlModel）及切片序列（layers）两个参数一直不变外，尝试改变后面几个参数（网格尺寸gridSize、危险面片临界角度crAngle、支撑到模型的横向间距xyGap），查看这些参数变化对支撑区域的影响。

2. 支撑区域识别

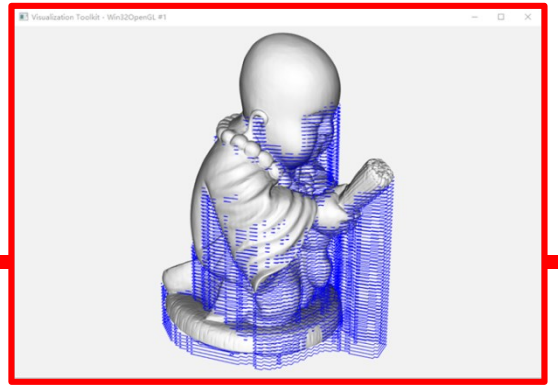
• 2.5. 支撑区域生成测试

- 左图是monk模型在第22层切片上的轮廓，包括模型轮廓和支撑轮廓，其中图a对应的参数为：gridSize = 2、xyGap = 1、crAngle = 60，单位为mm或角度，下面以该图为参照分析其他图形。
- 对比图a、b及c，当控制其他参数不变，只变化crAngle参数时，临界角度值设置的越大，最终得到的支撑区域面积越大。危险面片的定义是小于临界角的面片，当临界角值越大，意味着模型上将有越多的面片被判定为危险面片，需要添加支撑的范围自然就会变大。
- 对比图a和d，当控制其他参数不变，只变化gridSize参数时，网格尺寸越小，生成支撑区域的边界线就越精细，显然这对提高支撑区域的精度时有意义的，但前提是增加了计算量。可以估计，将gridSize值从2mm减小到1mm时，网格节点数增加了4倍，后续相应的求交计算量必然也成倍增加。考虑到打印时允许冗余支撑的存在，而且错误的支撑区域可以通过轮廓偏置和布尔运算的方式被修正，因此，在实际计算中，为了保证支撑区域生成的效率，不建议将gridSize值设置得过小。
- 对图e，设置xyGap值为2，此时支撑轮廓和模型轮廓之间的间距被拉大，这有利于后期将打印好的支撑材料从模型上脱离。但xyGap值不能过大，否则模型将处于欠支撑状态，不利于悬臂结构成型。xyGap的具体值和打印材料的属性有关，应该从大量的工艺试验中总结得出。
- 在图f中，在FindSptRegion类的execute函数（参见代码10-7）中将genSptRegions函数替换成了genRawSptRegions，即直接展示由切片支撑点影响区域（矩形）布尔并运算得到初始支撑区域。从该图可以看出，初始支撑区域确实可能会和模型区域在边界附近重叠，也进一步印证了在genSptRegions函数中添加轮廓偏置和布尔减运算的必要性。



monk模型某层切片上在不同参数设置下的支撑轮廓变化情况

2. 支撑区域识别



monk模型各层支撑轮廓

- **2.5. 支撑区域生成测试**

- **模型测试：** 下图给出了针对整个monk模型所有切片上的支撑轮廓（**蓝色**），结合该图可看到支撑轮廓的全貌。在monk模型的底座、袖口、手、香、下巴以及微微前倾的脸部等肉眼判断需要添加支撑的部位，都已经被添加上了支撑，可见findSptRegion函数确实实现了它应用的功能。
- **效率测试：** **（参照轮廓路径生成函数genCpPath）**
- 当切片厚度为2.0mm时，支撑轮廓生成时间是3.9秒，此时生成模型轮廓路径的时间为13.1秒；当切片厚度为0.2mm时，支撑轮廓生成时间是14.9秒，此时生成模型轮廓路径的时间为132.9秒。
- 一方面，对相同SLC文件，findSptRegion比genCpPath运行更快；另一方面，对findSptRegion，当切片层数增加10倍时，运行时间只增加了约4倍，可见findSptRegion的运行效率受切片厚度减小（即切片数量增加）的影响较小，敏感度不高。

monk模型在不同切片厚度下的支撑轮廓生成时间统计

模型	面片数	模型高度 (mm)	切片厚度 (mm)	层数	支撑轮廓生成时间 (秒)
monk	336,974	175.5	2.0	87	3.9
			1.0	175	4.7
			0.5	351	7.2
			0.2	877	14.9

主机CPU为intel Core i7-8700 @ 3.20GH　　gridSize = 2、xyGap = 1、crAngle = 60

3. 支撑路径生成

FindSptRegion 模块

针对给定的三维模型及其切片数据

在各层上生成相应的支撑区域

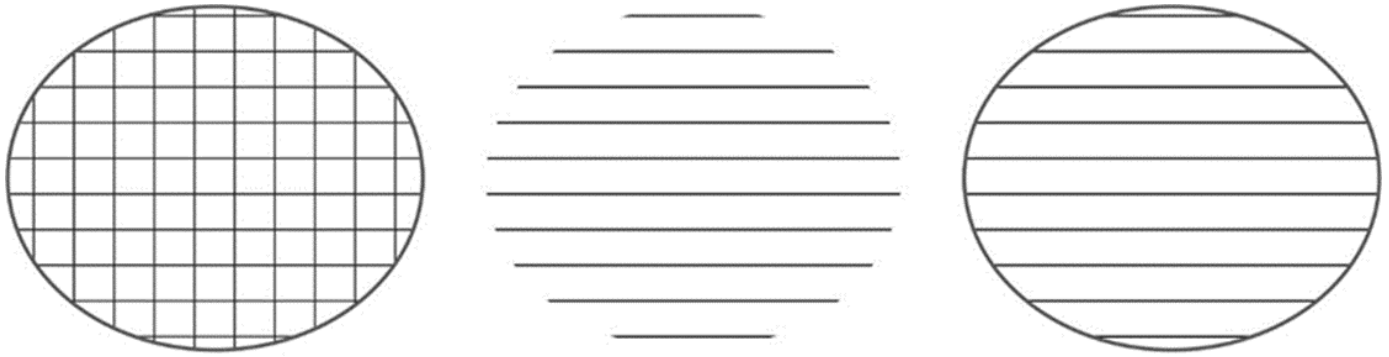
任 务

在得到的支撑区域内

填充支撑路径

考虑到支撑部分对轮廓精度要求不高，为保证填充效率，应尽量少用或不用轮廓路径，**可以平行路径填充为主。**

- 下图给出了支撑填充路径的主要形式，即区域内部以平行路径填充，在区域边界，填充一圈（或几圈）轮廓路径。
- 填充轮廓路径的目的是加固支撑强度，避免支撑材料在高度增长的时候倒塌，特别是当支撑区域内部的上下层的填充角度保持不变时。
- 相对于模型部分的填充路径，支撑部分平行路径之间的**间距可以设置得更大**，这样有利于后期将支撑从模型上剥离。



(a) 轮廓+平行交叉填充　(b) 平行同向填充　(c) 轮廓+平行同向填充

3. 支撑路径生成

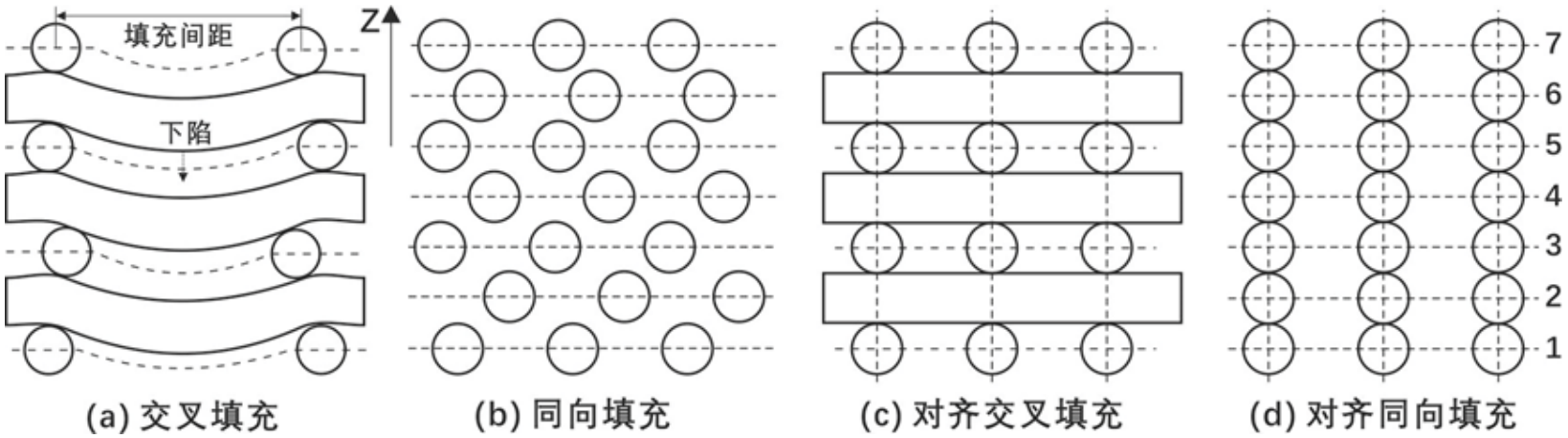
• 3.1. 平行填充扩展

- **回顾：**第9章封装了一个名为GenDpPath的类（对应接口函数为genDpPath），用于生成指定平面区域内的平行填充路径（包括平行线之间的连接）。

GenDpPath 类能否直接用于生成支撑区域内的平行填充路径呢？——不能

GenDpPath类设计时的默认了两个条件：

- 模型切片上下层之间呈正交（或一定的角度差）填充，以形成上下层之间的编织效果，提升打印强度；
- 同一层切片上填充间距不大，打印过程中上层打印材料在下层填充缝隙间会稍有下陷变形，但不至于影响整体打印效果（图a）。



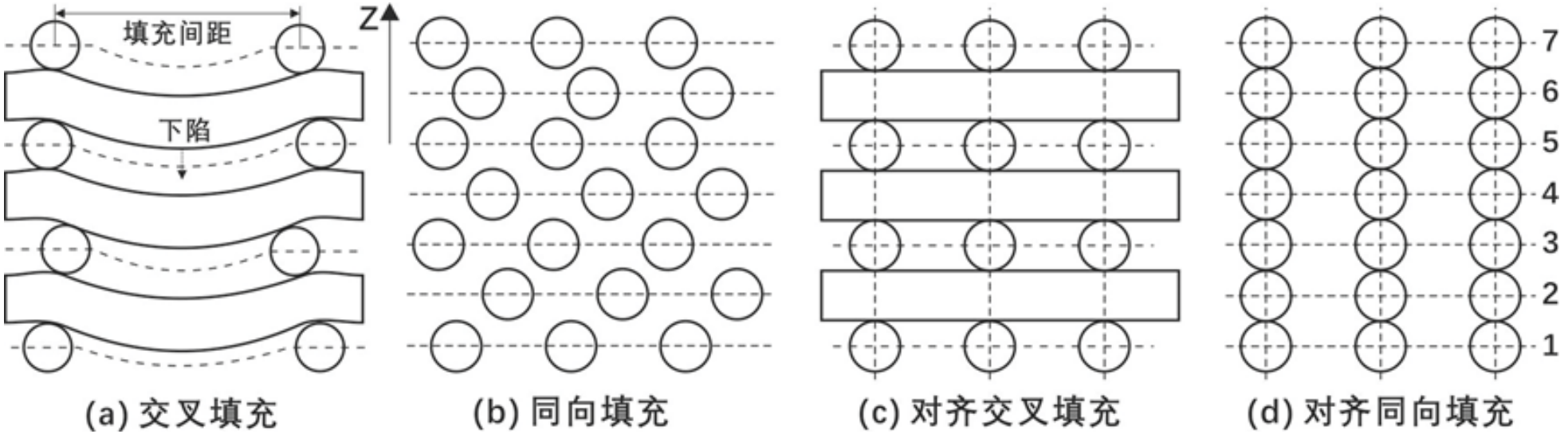
(a) 交叉填充　(b) 同向填充　(c) 对齐交叉填充　(d) 对齐同向填充

上下层材料交叉和对齐填充关系

3. 支撑路径生成

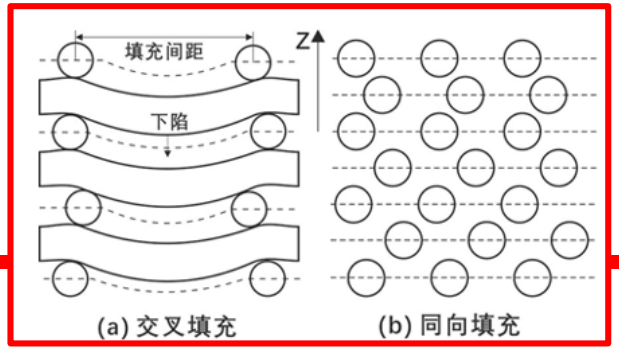
• 3.1. 平行填充扩展

- 在以上两个默认条件下，之前在设计 GenDpPath 类时**并没有过多地考虑上下层平行线（路径）之间的对齐问题**。
- 理想情况下，隔层（如图c）或相邻层（如图d）**平行填充线应尽量对齐**，特别是对于支撑部分填充路径。考虑支撑填充路径间距大，且可能采用同向填充策略，此时如果上下层填充线不对齐，则在实际打印过程中将因上层材料无法依附而导致打印失败（如图b）。
- 根据以上分析，如果仍然使用GenDpPath类来生成支撑填充路径，那么需要**对GenDpPath类做一些维护和升级工作**。



上下层材料交叉和对齐填充关系

3. 支撑路径生成



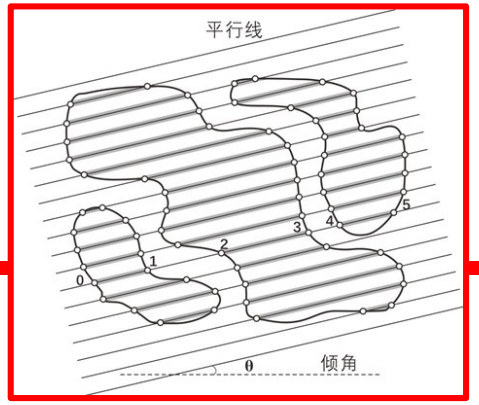
• 3.1. 平行填充扩展

- GenDpPath类的流程控制函数 **generate**（参见代码9-12）用 **ys** 变量控制当前切片层上的平行线位置。
  - 在generate函数中，对每个切片层，都会根据当前层填充区域生成一个独立的ys，这是导致上下层平行路径间无法对齐的根本原因。
  - 为对齐上下层路径，只需在generate函数中对所有切片层使用一个唯一的 ys 即可，而这个唯一的ys可以根据三维模型的极限坐标生成。
- 根据这一原理，在generate函数基础上，在GenDpPath类中添加了一个功能相似的函数**generateEx**，具体实现如下：——ys从外部输入，兼容generate函数！

GenDpPath.py	对GenDpPath类进行扩展和升级，添加generateEx函数	10-9

```
1   class GenDpPath:
2     def generateEx(self, ys = None, center = None): # 定义generateEx函数
3       rotPolys = GeomAlgo.rotatePolygons(self.polygons, -self.angle, center)   # 旋转多边形
4       if ys is None:                                  # 如果参数ys为None
5         ys = self.genScanYs(rotPolys)                 # 则根据多边形生成独立的ys
6       self.splitPolys = splitRegion(rotPolys)         # 分区生成单连同区域
7       paths = []                                      # 定义路径序列
8       for poly in self.splitPolys:                    # 对每个单连通区域生成路径
9         segs = genHatches([poly], ys)                 # 生成平行填充线段
10        if len(segs) > 0:
11          path = self.linkLocalHatches(segs)          #连接单连通区域内填充线段
12          paths.append(path)
13      return GeomAlgo.rotatePolygons(paths, self.angle, center)          # 反向旋转并返回路径
14  def genDpPathEx(polygons, interval, angle, ys = None, center = None):   # generateEx接口函数
15    return GenDpPath(polygons, interval, angle).generateEx(ys, center)
```

3. 支撑路径生成



• 3.1. 平行填充扩展

- **generateEx**函数有两个输入参数，分别为 ys 和 center：
  - **ys**　　用来控制当前切片层上平行线位置和间距的实数序列，该参数由**外部生成和输入**可确保对所有层切片来说，ys是唯一的。
  - **Center**　旋转中心点

为什么要在 generateEx 函数中设置旋转中心点 center?

- ✓ 在generate函数中，为了适应平行路径不同填充角度的需求，根据相对运动的原理需要对表示打印区域的多边形轮廓进行旋转，在完成填充路径计算后再将路径旋转相反的角度。
- ✓ 在generate函数中，因为ys根据旋转后的多边形轮廓产生，因此理论上ys对应的平行线总能覆盖旋转后的多边形区域，且旋转中心点是可以任意选择的（简单起见，generate函数选择了平面坐标系原点为旋转中心点）。
- ✓ 但是对generateEx函数来说，ys从外部输入，在多边形轮廓旋转之前已经生成，理论上，ys对应的平行线和旋转后的多边形并无关系，为了达到填充的目的，有必要通过外部措施来保证ys对应的平行线能否覆盖旋转后的多边形区域，而且是所有切片层上的旋转多边形区域。
- ✓ 采取的外部措施包括两点：一是根据模型沿Z方向的外接圆柱来生成ys，二是以**外接圆柱轴线沿Z轴投影点为旋转中心点**。通过这两个措施，在任意切片层上，无论多边形轮廓如何旋转，总能保证ys对应的平行线能够覆盖多边形区域。

genDpPathEx 函数完全兼容 genDpPath 函数!

3. 支撑路径生成

- **3.2. 支撑路径生成实现**

- 在完成GenDpPath模块的扩展和升级之后，下面给出针对支撑区域的填充路径生成代码，写在**GenSptPath模块**中，包括一个全局函数genSptPath和一个枚举类SptPathType，具体实现如下：

GenSptPath.py　实现支撑路径生成函数genSptPath　10-10

```
1   ……                                                    # 导入相关库
2   class SptFillType(Enum):                              # 填充路径形式枚举类
3       line = 1                                          # 平行同向填充
4       cross = 2                                         # 平行交叉填充
5   # 定义支撑路径生成函数
6   def genSptPath(stlModel, layers, pathInvl, gridSize, crAngle, fillType, fillAngle = 0, xyGap = 1):
7       findSptRegion(stlModel, layers, gridSize, crAngle, xyGap)  # 生成支撑区域
8       xMin, xMax, yMin, yMax, zMin, zMax = stlModel.getBounds()          # 计算模型在各个方向极限
9       center = Point3D((xMin + xMax)/2, (yMin + yMax)/2)# 模型在平面上中心点
10      corner = Point3D(xMax, yMax)                      # 模型包络在平面上角点
11      radius = center.distance(corner)                  # 计算模型外接圆柱半径
12      ys, y = [], center.y - radius
13      while y <= center.y + radius:                     # 生成实数序列ys
14          y += pathInvl
15          ys.append(y)
16      for i, layer in enumerate(layers):                # 遍历每层切片
17          angle = fillAngle                             # 填充角度
18          if fillType == SptFillType.cross and i % 2 == 1:   # 如果是平行交叉填充,
19              angle = fillAngle + math.pi/2             # 计算交叉填充角度
20          layer.sptDpPaths = genDpPathEx(layer.sptContours, 0, angle, ys, center)  # 平行填充路径
21          layer.sptCpPaths = layer.sptContours                    # 外周轮廓路径
```

1. 求支撑区域

2. 求ys和center

3. 求填充路径

3. 支撑路径生成

- **3.2. 支撑路径生成实现**
- **SptFillType** 枚举类，它枚举了目前我们能够实现的两种支撑路径形式：



- **genSptPath** 函数共有8个输入参数：
  - **stlModel** **表示三维模型**
  - **Layers** **为对应的切片数据**
  - **pathInvl** **为相邻平行路径之间的间距**
  - **gridSize** **为XY平面上的预设网格尺寸**
  - **crAngle** **为危险面片临界角**
  - **fillType** **为SptFillType枚举类型的支撑路径填充形式**
  - **fillAngle** **表示填充角度（默认值为0度）**
  - **xyGap** **为支撑到模型的横向间隙（默认值为1mm）**

- ✓ stlModel、layers、gridSize、crAngle、xyGap 这 5 个参数和支撑区域生成相关，将直接被 findSptRegion 函数所用。
- ✓ 其他参数和路径生成相关。

3. 支撑路径生成

- **3.2. 支撑路径生成实现**

- **genSptPath**函数在流程上主要可以分为三个步骤：

  ① 调用findSptRegion函数生成了各层切片上的**支撑区域轮廓**，这些支撑轮廓被保存在layers序列中。

  ② 计算模型在XY平面上的中心点**center**和全局唯一的用于指示平行线位置和间距的实数序列**ys**。

  ③ 调用genDpPathEx函数对每层切片上的支撑区域生成**平行填充路径**。

- 在genSptPath函数中，将得到的平行填充路径保存到了每层切片（layer）的sptDpPaths序列中，并且直接以平行填充路径外围的支撑区域轮廓（sptContours）为轮廓路径，保存到每层切片的sptCpPaths序列中。

- **注意：**以上sptDpPaths和sptCpPaths两个序列（其实也包括sptContours序列）都是Python语言特有的动态属性，如果使用其他编程语言实现，应考虑提前在Layer类中定义相应属性。

- genSptPath函数的最终**输出**就是sptDpPaths和sptCpPaths两个路径序列，其中对sptDpPaths中的平行填充路径来说，它们已经具备了分区特性，且各个分区内的路径都已经被连接。

3. 支撑路径生成

• 3.3. 支撑路径生成测试

- 最后对得到的支撑路径生成函数**genSptPath**做一个简单的测试，看它是否能够生成符合要求的支撑填充路径，包括同向填充和交叉填充。方便起见，直接将测试代码写在GenSptPath模块的main函数中，具体如下：

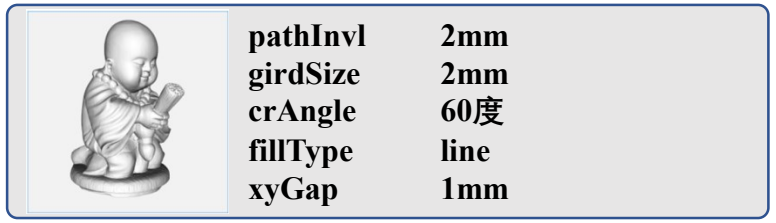
GenSptPath.py | 在模块main函数中对genSptPath函数进行测试 | 10-11

```
1   ……                                          # 导入相关库，忽略其他代码
2   if __name__ == '__main__':
3       src, stlModel = vtk.vtkSTLReader(), StlModel()
4       src.SetFileName("E:\\Models\\STL\\monk.stl")      # 读取STL模型文件
5       stlModel.extractFromVtkStlReader(src)             # 提取VTK数据源
6       layers = readSlcFile("D:\\monk at 2.0mm.slc")     # 读取相应切片文件
7       genSptPath(stlModel, layers, 2, 2, degToRad(60), SptFillType.line)
8       va = VtkAdaptor()                                 # 以下调用VTK显示计算结果
9       va.drawPdSrc(src).GetProperty().SetOpacity(0.1)   # 显示三维模型
10      for layer in layers:                              # 逐层显示填充路径
11          for poly in layer.sptDpPaths:                 # 显示平行填充路径
12              if poly is not None:
13                  va.drawPolyline(poly).GetProperty().SetColor(1, 0, 0)    # 平行路径红色
14          for poly in layer.sptCpPaths:                 # 显示轮廓路径
15              if poly is not None:
16                  va.drawPolyline(poly).GetProperty().SetColor(0, 0, 1)    # 轮廓路径蓝色
17      va.display()
```

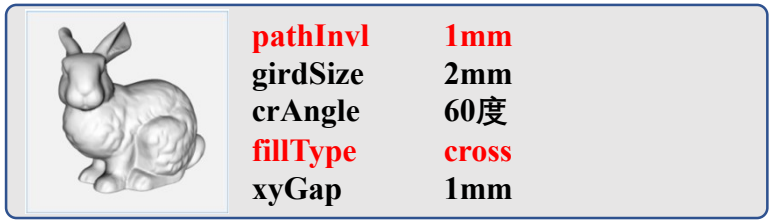
3. 支撑路径生成

• 3.3. 支撑路径生成测试

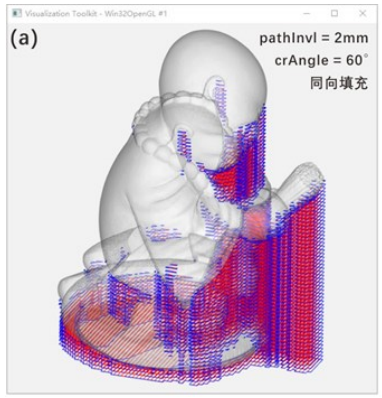
- 选择 monk 和 bunny 两个模型为测试模型，对应的切片文件切片厚度均为2.0mm。

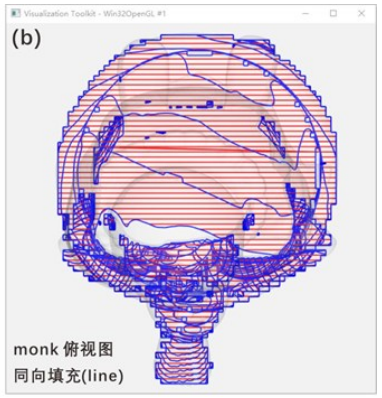


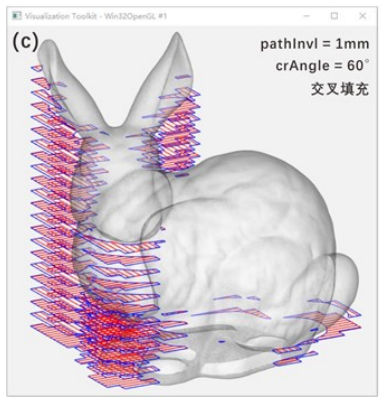
pathInvl	2mm
girdSize	2mm
crAngle	60度
fillType	line
xyGap	1mm

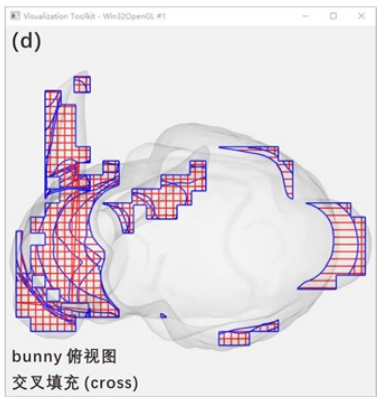


pathInvl	1mm
girdSize	2mm
crAngle	60度
fillType	cross
xyGap	1mm









- **测试结论：** genSptPath函数能够根据设置的参数在模型上生成完备的支撑路径，这得益于对FindSptRegion类的良好封装；无论是同向填充路径还是交叉填充路径，相邻上下切片层之间的平行路径都已严格对齐，这得益于对GenDpPath模块的扩展和改进。
- **效率测试：** 对monk模型，切片厚度为0.2mm，其他参数如上，测试平台CPU为intel Core i7-8700 @ 3.20GH，genSptPath函数的执行时间约为**30秒**。

4. 支撑量优化

支撑路径总长度 → **决定** → **支撑材料用量**

打印完成后，支撑材料是需要被去除并舍弃的。

结论：在保证支撑有效且强度足够的情况下，支撑量越少越好。

在相同的支撑参数设置下，对同一模型，支撑量只和模型在工作台上的摆放角度有关

路径间距pathInvl、面片临界角crAngle、填充形式fillType、填充角度fillAngle、横向间距xyGap等

措施：

为了降低支撑量，考虑使用一种智能优化算法对模型的**摆放角度**进行优化。

智能优化算法

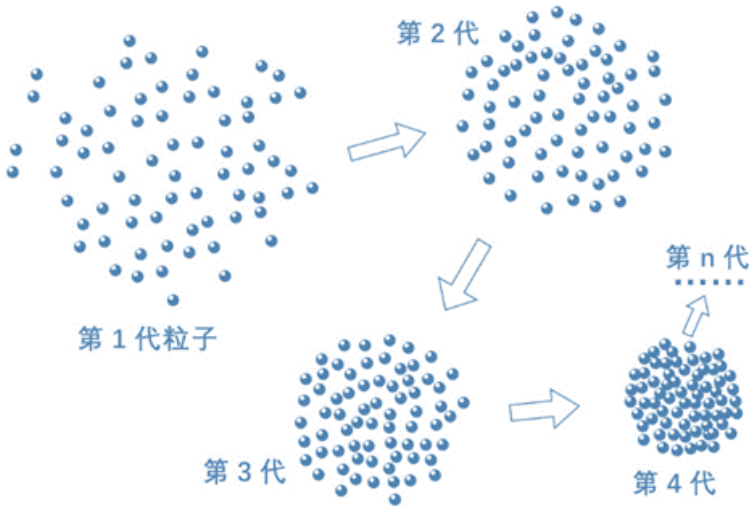
- ✓ 遗传算法、粒子群算法、免疫算法、模拟退火算法、鱼群算法、蚁群算法等，这些算法大都受自然界生物群体行为或物理现象的启发，依赖“**随机—迭代—进化**”过程，在解空间中搜索最优解。
- ✓ 在这些算法中，**粒子群算法**原理相对简单易懂，代码实现相对容易，因此，下面将采用粒子群算法对模型摆放角度进行优化。

4. 支撑量优化

• 4.1. 粒子群算法原理

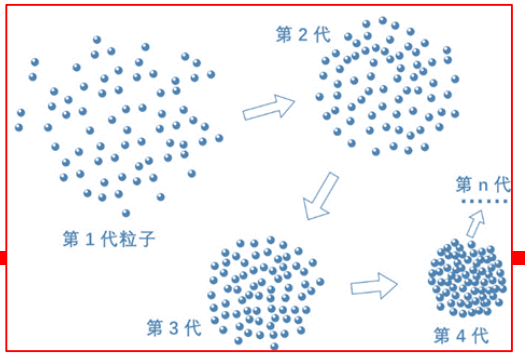
粒子群算法（Particle Swarm Optimization，PSO）最早由美国两位科学家Eberhart和kennedy在1995年提出。它源于对鸟类集群捕食行为的研究，利用群体中个体对信息的共享使整个群体的运动在问题求解空间中产生从无序到有序的演化过程。

- ✓ 和遗传算法类似，粒子群算法也是用**适应度（fitness）**来评价解的品质，从一群随机粒子（随机解）出发，通过迭代寻找最优解。
- ✓ 在每次迭代中，每个粒子通过跟踪两个“极值”更新自己，一个是该粒子本身所找到的最优解，叫做**个体极值**；另一个是从整个群体中当前找到的最优解，叫做**全局极值**。
- ✓ 相比于遗传算法，粒子群算法更为简单，易实现且仅有少量参数需要调整。粒子群算法通常被用于求解组合优化问题的最优解。



粒子群算法粒子演化收敛过程示例

4. 支撑量优化



• 4.1. 粒子群算法原理

在粒子群算法中，每个粒子都有一个随机产生的初始**位置**和**速度**，并且有一个通过求解适应度函数得到的**适应度**来评价该粒子在当前位置上的**品质**。粒子群算法实现的**基本步骤**如下:

步骤1. 随机初始化N个粒子，包括每个粒子的位置Pi和速度Vi，其中i ∈ [1, N]；

步骤2. 根据指定适应度函数计算每个粒子适应度；

步骤3. 比较粒子群中每个粒子Pi和与其对应的个体极值LPi的适应度，如果Pi优于LPi，则用Pi替换LPi；

步骤4. 比较粒子群中每个粒子Pi和全局极值GP的适应度，如果Pi优于GP，则用Pi替换GP；

步骤5. 根据下列进化公式对每个粒子速度进行操作并移动粒子，

$$V_i(t+1) = \omega \cdot V_i(t) + r_1\left(LP_i - P_i(t)\right) + r_2 \cdot (GP - P_i(t)), \qquad (10\text{-}5)$$

$$P_i(t+1) = P_i(t) + V_i(t+1), \qquad (10\text{-}6)$$

其中 t 为当前迭代次数，ω 为粒子惯性权重，是粒子自我价值实现的体现，r_1、r_2 分别为粒子向个体极值和全局极值进化的学习因子，ω、r_1、r_2 一般是随机生成的取值为[0, 1]范围内的实数。

步骤6. 如果停止条件满足，则跳出循环，否则跳至步骤2。

以上步骤中，对每个粒子，**位置**、**速度**及**适应度函数（fitness function）**的定义是整个粒子群算法中最为基础的环节，特别是适应度函数。

粒子的适应度是从某个角度衡量粒子品质的一种指标，而适应度函数就是计算粒子适应度的一种方法。

在粒子群算法中，一般会按照被求解问题本身的**目标函数**去定义粒子的**适应度函数**。

对支撑量优化问题，目标函数就是**支撑量关于模型摆放角度的函数**，而求解目标函数的最优解就是寻找支撑量最少时的模型摆放角度，即最优模型摆放角度。

4. 支撑量优化

- **4.2. 适应度函数建立：支撑量关系分析**

在控制其他变量一致的情况下，模型上的**支撑量只和模型的摆放角度**有关，而与模型的摆放位置无关。

几何上，一个模型的摆放角度可由该模型绕坐标系 X 轴、Y 轴和 Z 轴的旋转角度来定义。

- ✓ 这里的坐标系可以是世界坐标系，也可以是在模型上建立的某个局部坐标系。
- ✓ 考虑到模型平移支撑量的不变性，简单起见，这里不再定义其他坐标系，就以世界坐标系的各坐标轴为转轴去旋转模型。

模型沿着坐标系 Z 轴方向进行打印，绕 Z 轴对模型进行旋转并不会改变模型上的支撑空间形状和支撑量，而只会改变支撑位置。

- ✓ 因为模型绕Z轴旋转时模型上各个面片相对于XY平面的倾角事实上并未发生改变。

结论：模型上的支撑量只和模型绕X轴和Y轴的旋转角度有关

4. 支撑量优化

模型在不同摆放角度下的支撑量 ← 对模型按指定角度进行旋转变换并获取其在旋转后的坐标

• 4.2. 适应度函数建立：模型旋转变换

- 记模型绕坐标系X轴的旋转角为α，绕Y轴的旋转角为β，α、β的定义域均为[0, 2π]，初始状态下（即模型刚被加载时），α = 0，β = 0。
- 分别构建绕X、Y两轴旋转的两个旋转矩阵 $M_{rx}(\alpha)$、$M_{ry}(\beta)$。对任一三角面片上的一个顶点 P 及法向量 N，可以根据下式对其进行旋转变换：

$$P' = \mathrm{P} \cdot M_{rx}(\alpha) \cdot M_{ry}(\beta), \qquad (10\text{-}7)$$

$$N' = \mathrm{N} \cdot M_{rx}(\alpha) \cdot M_{ry}(\beta), \qquad (10\text{-}8)$$

- 根据式(10-7)和(10-8)，遍历STL模型中每个面片的顶点和法向量，完成对模型的旋转和坐标更新：

StlModel.py	在StlModel类中添加旋转变换函数	10-12

```
1   class StlModel:
2       def multiplied(self, m):                    # 根据矩阵进行几何变换
3           model = StlModel()                      # 新建一个模型
4           for t in self.triangles:                # 遍历每个面片
5               triangle = Triangle(t.A * m, t.B * m, t.C * m, t.N * m)  # 对顶点及法矢作用矩阵
6               model.triangles.append(triangle)
7           return model                            # 返回模型
8       def rotated(self, a, b, c):                 # 指定角度对模型旋转变换
9           mx = Matrix3D.createRotateMatrix('X', a)    # 绕X轴的旋转矩阵
10          my = Matrix3D.createRotateMatrix('Y', b)    # 绕Y轴的旋转矩阵
11          mz = Matrix3D.createRotateMatrix('Z', c)    # 绕Z轴的旋转矩阵
12          m = mx * my * mz                        # 矩阵点乘的积
13          return self.multiplied(m)               # 几何变换
```

以矩阵为参数，可在函数外部创建变换矩阵，对STL模型进行平移、旋转、缩放等几何操作

特化函数，专门用于旋转STL模型，接受绕X、Y、Z三轴旋转的角度a、b、c为参数，角度单位为弧度

4.支撑量优化

- **4.2. 适应度函数建立：长度 $L_s(\alpha,\beta)$**

- 对旋转后新模型（对应旋转角度分别为α、β），可将其输入 **genSptPath** 函数计算 **支撑路径总长度**（支撑量的一种表征），如果记总长度为 L_s，那么就可以建立 L_s 关于转角 α、β 的映射（函数）关系：

$$L_s = L_s(\alpha,\beta). \qquad (10\text{-}9)$$

目标函数：寻找最优的转角 α 和 β 值，使得支撑路径总长度 L_s 最小，即支撑量最小。

- 针对式(10-9)，可非常直观地对粒子群算法中粒子、适应度等概念进行定义。
  - ✓ **粒子** 一个由 α、β 构成的转角对
  - ✓ **适应度** 支撑路径总程度 L_s
  - ✓ **适应度函数** $L_s(\alpha,\beta)$
  - ✓ **粒子的位置** 由 α、β 构成的角度坐标 (α, β)

4.支撑量优化

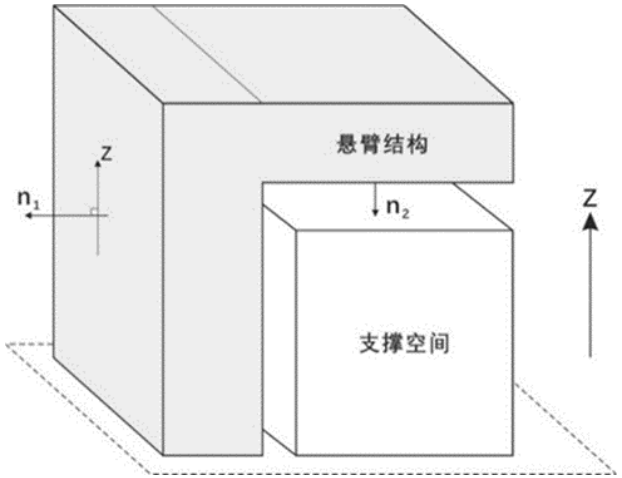
- **4.2. 适应度函数建立：体积 $V_s(\alpha, \beta)$**

> **□ 选择$L_s(\alpha, \beta)$做适应度函数是否一个明智的选择?**
>
> - 如果要计算支撑路径总长度L_s，必须在 **genSptPath** 函数中输入模型（stlModel）和相应的模型切片序列（layers）等参数，也就是必须首先对模型进行切片。
> - 一方面，切片操作消耗的时间比较可观；
> - 此外，切片厚度如何设置？**评估模型外的支撑量却要对模型本身切片——不合逻辑!**

- 适应度函数将被**非常频繁**地调用，一个好的适应度函数应该是**几何意义明确**且**计算效率较高**的函数。因此，有必要寻找一个计算量更小的适应度函数。

> **□ 更合理的支撑量评估标准——支撑体积**
>
> - 和切片过程不相关，但能反应支撑量的概念，即处在危险面片正下方、模型外部的空间体积。
> - 可用支撑空间体积 V_s 替换支撑路径总长度 L_s，作为粒子的适应度，即：

$$V_s = V_s(\alpha, \beta). \qquad (10\text{-}10)$$



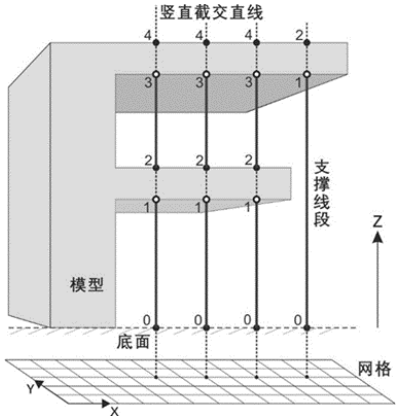
4. 支撑量优化

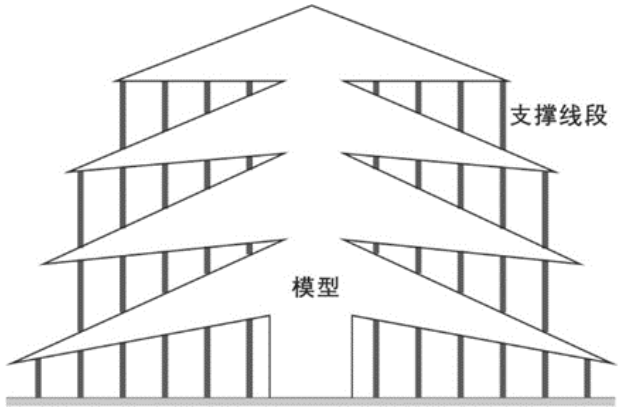
- **4.2. 适应度函数建立：如何计算支撑体积 Vs?**

- 在封装 **FindSptRegion** 类时，已涉及支撑体积计算的相关概念——**支撑线段**。为了降低计算量，通过合理设置网格尺寸，让支撑线段稀疏地分布在支撑空间内。

支撑体积 = 单元网格面积 × 支撑线段总长度

- **体积精度**和投影网格精细程度有关，**网格尺寸越小**，数量越多，得到的支撑空间**体积越精确**。





说明

- FindSptRegion类没有直接给出获取支撑线段（长度）的相关代码，但通过**calcModelSptPoints**函数可计算过网格结点的竖直线和模型的交点——模型支撑点，且该函数已经过优化，具有较高的执行效率。
- 基于模型支撑点，可以利用**交点序号的奇偶属性来提取支撑线段**，即位于偶数序号和奇数序号之间的线段为支撑线段。

4. 支撑量优化

• 4.2. 适应度函数建立：代码

- 根据上述分析，下面以模型上的支撑空间体积为粒子的适应度，以模型绕X、Y轴的转角对 (α, β) 为粒子位置坐标，给出粒子**适应度定量计算的函数**，函数名为**fitness**。这部分代码写在一个新建的 **PsoModelPos** 模块中，具体实现如下：

PsoModelPos.py	粒子群算法优化模型摆放角度模块：定义粒子适应度函数	10-13

```
1   ……                                          # 导入相关库
2   def fitness(stlModel, a, b, gridSize, crAngle):   # 定义适应度函数
3       sptLength = 0.0                         # 支撑线段总长度
4       newModel = stlModel.rotated(a, b, 0)    # 对模型绕X、Y轴旋转变换
5       fsr = FindSptRegion(newModel, None, gridSize, crAngle) # 定义FindSptRegion对象fsr
6       gridDic = fsr.calcModelSptPoints()      # 模型支撑点，返回gridDic
7       for key in gridDic:                     # 根据交点奇偶性和crAngle
8           zas = gridDic[key]                  # 提取支撑线段
9           for i in range(0, len(zas), 2):
10              if i + 1 <= len(zas) - 1:
11                  za0, za1 = zas[i], zas[i+1]
12                  if za1[1] < crAngle:
13                      sptLength += (za1[0] - za0[0])   # 累加支撑线段长度
14      return sptLength *fsr.ax*fsr.ay         # 计算并返回支撑空间体积
```

- fitness函数共有5个输入参数
  - ✓ **stlModel** 为三维模型
  - ✓ **a和b** 为模型绕X、Y轴的旋转角度（单位为弧度）
  - ✓ **gridSize** 模型投影网格尺寸
  - ✓ **crAngle** 为危险面片临界角（单位为弧度）。
- **必须强调的是，这里的stlModel为未经任何几何变换的初始模型。**

4. 支撑量优化

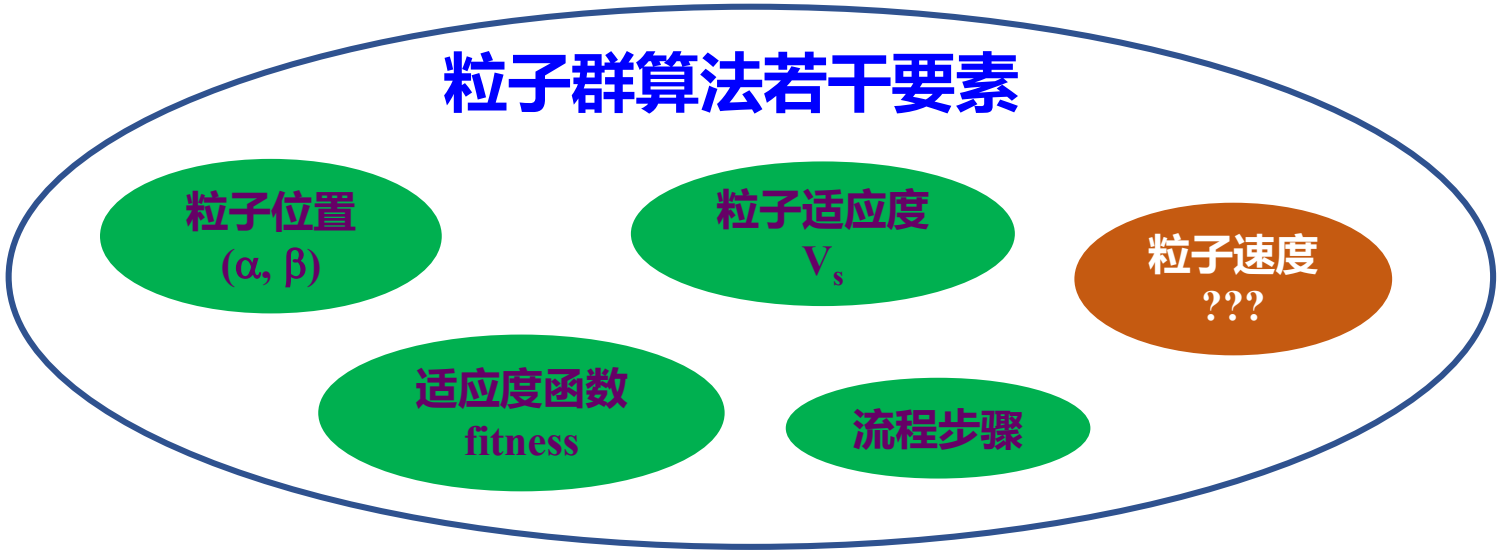
• 4.2. 适应度函数建立：代码分析

- 在fitness函数中，首先根据转角参数a和b对输入模型进行了旋转变换，得到了一个**新模型**。
- 然后我们以新模型以及网格尺寸、临界角作为参数构造了一个FindSptRegion类的对象fsr，注意，在构造该对象时，对FindSptRegion的layers参数赋了None值，表明这个参数是不需要的，这也正是用支撑空间替换支撑路径长度作为粒子适应度的原因。
- 随后通过调用**calcModelSptPoints**函数计算并获得了模型支撑点，且这些点被组织并存放在了字典gridDic中，其中字典键为网格节点坐标，字典值为过当前网格节点同一竖线上的支撑点。通过gridDic对象，可以根据支撑点的序号的奇偶性及相应面片的临界角来提取支撑线段并计算其长度。
- 最后结合实际单元网格边长（FindSptRegion类中保存的ax和ay值），可以计算支撑空间的体积。

- **说明：** fitness函数涉及的**计算量**仍然是**相当可观**的。它的主要计算量消耗在对模型和旋转变换和对 **calcModelSptPoints** 函数的调用中，因为这两处地方都需要对模型中的所有面片进行遍历，而这些遍历是不可避免的。

4. 支撑量优化

- **4.3. 粒子定义**



$$V_i(t+1) = \omega \cdot V_i(t) + r_1\left(LP_i - P_i(t)\right) + r_2 \cdot (GP - P_i(t)), \qquad (10\text{-}5)$$

$$P_i(t+1) = P_i(t) + V_i(t+1), \qquad (10\text{-}6)$$

✓ 根据式(10-5)、(10-6)可知，粒子的速度属性是粒子位置变化的原因。

✓ 如果粒子没了速度，静止不动，那粒子也就**不会进化**!

4. 支撑量优化

粒子速度
???

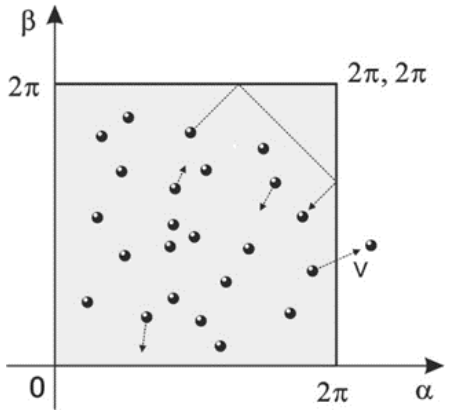
• 4.3. 粒子定义：速度定义难点

在当前应用场景下，对粒子的速度给出定义既简单又困难！

- **考虑到用转角对(α, β)表示粒子位置，理论上，转角增量(Δα, Δβ)就是粒子的速度。**

□ 粒子“越界困境”

- 如果以模型绕 X 轴的转角 α 为横坐标、绕 Y 轴的转角 β 为纵坐标建立平面正交坐标系，考虑到转角 α、β 的**定义域**均为 [0, 2π]，那么粒子在平面上的活动范围为一个边长为2π的正方形区域，如图所示。
- 在这个场景下，粒子**进化的本质**是粒子的位置叠加一个速度矢量。假如在某一轮迭代中，某个粒子恰好位于正方形区域边界附近，而它此时的速度指向正方形区域外侧，那么在这个粒子进化过程中，很有可能“跳”出正方形区域。



模型转角坐标系及粒子定义域

- **必须采取必要的手段，将粒子的进化控制在定义域内。**

- **方案一**

仿照物理中的“碰撞反射”概念，让粒子在碰到定义域边界时，反射回定义域内部——这样做的难点在于粒子反射路线的建模与计算，特别是当粒子速度较大，可能需要多次反射才能静止，如图所示。

- **方案二**

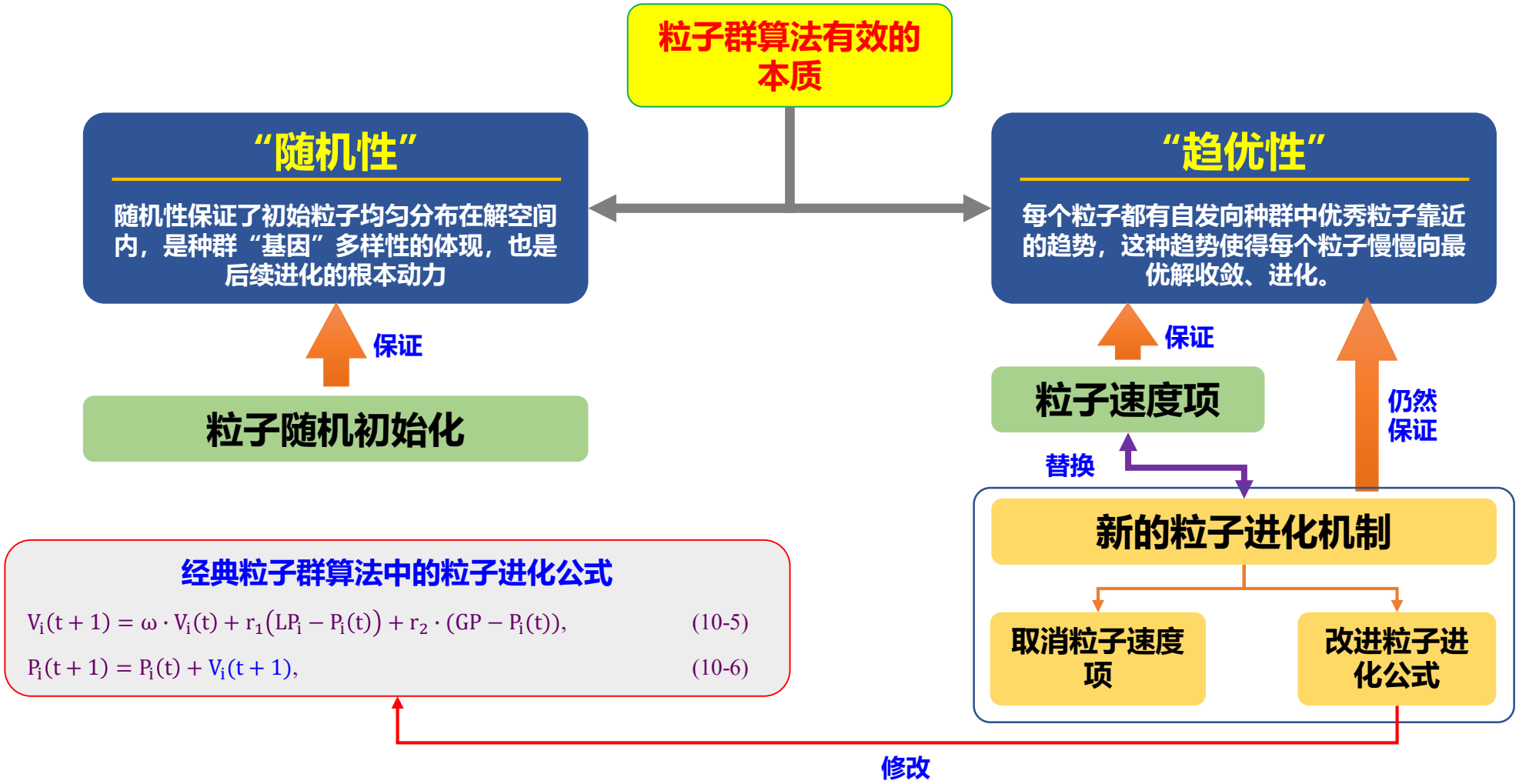
根据转角的周期性，当转角值超过定义域时，通过加减2π的整数倍，直接将其拉回定义域内——这种方法的缺点是几何意义不明确，可能存在算法收敛性问题。

结论：在当前应用背景下，定义粒子的速度项非常困难！

4. 支撑量优化

粒子速度
???

- 4.3. 粒子定义：深入分析



经典粒子群算法中的粒子进化公式

$$V_i(t+1) = \omega \cdot V_i(t) + r_1\left(LP_i - P_i(t)\right) + r_2 \cdot (GP - P_i(t)), \quad (10\text{-}5)$$

$$P_i(t+1) = P_i(t) + V_i(t+1), \quad (10\text{-}6)$$

4. 支撑量优化

粒子群算法中的3种不同粒子

普通粒子P_i	个体极值粒子LP_i	全局极值粒子GP
这种粒子是参与迭代进化的主体，粒子数量（即种群数量）根据人为需求设定	这种粒子是普通粒子的伴随粒子，和普通粒子存在一一对应的关系，数量与普通粒子相当	这种粒子全局只有一个

• 4.3. 粒子定义：改进进化机制

- 根据粒子群算法基本原理，每个普通粒子 P_i 都有向对应的个体极值 Lp_i 以及全局极值 GP 靠近的趋势。在取消了粒子速度项、仅保留粒子位置项的情况下，可使用下式来描述普通粒子向个体极值和全局极值的靠近趋势：

$$\mathbf{P_i(t+1) = p \cdot P_i(t) + q \cdot LP_i(t) + r \cdot GP,} \qquad \textbf{(10-11)}$$

式中i为粒子序号，t为迭代次数，p、q、r为各粒子权重系数，取值范围为[0, 1]随机数，且满足：

$$\mathbf{p + q + r = 1.} \qquad \textbf{(10-12)}$$

其中 p 为粒子的惯性系数，是粒子实现自我价值的体现，q、r 分别为粒子向个体极值和全局 极值进化的学习因子。为确保其和为1，p、q、r 具体可由下式计算：

$$\begin{cases} \mathbf{p} = \dfrac{r_1}{r_1 + w_L \cdot r_2 + w_G \cdot r_3} \\ \mathbf{q} = \dfrac{w_L \cdot r_2}{r_1 + w_L \cdot r_2 + w_G \cdot r_3}, \\ \mathbf{r} = \dfrac{w_G \cdot r_3}{r_1 + w_L \cdot r_2 + w_G \cdot r_3} \end{cases} \qquad \textbf{(10-13)}$$

式中r_1、r_2、r_3为三随机数；w_L、w_G也是两个随机数（权重系数），取值范围均为$[0, +\infty)$，用来调节个体极值和全局极值的相对重要性。

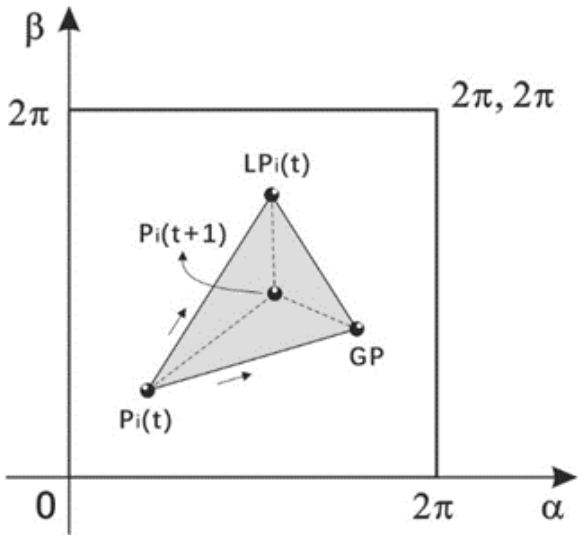
注意：式(10-11)和式(10-13)涉及双重嵌套随机权重，总体而言，式(10-11)中p、q、r三个权重系数的相对关系是由w_L、w_G两个权重系数决定的，它们之间的比例关系为p : q : r = 1 : wL : wG。

4. 支撑量优化

$$P_i(t+1) = p \cdot P_i(t) + q \cdot LP_i(t) + r \cdot GP, \qquad (10\text{-}11)$$

• 4.3. 粒子定义：公式(10-11)的几何和进化意义

- 如图，如果新的粒子 $P_i(t+1)$ 由式(10-11)计算得到，且满足式(10-12)，那么在转角坐标系平面上，$P_i(t+1)$ 必然落在由 $P_i(t)$、$LP_i(t)$、GP 三点确定的**三角形内**（包括边界）。
  - 这点非常有意义，预示着如果 $P_i(t)$、$LP_i(t)$、GP 三个粒子都在正方形定义域内，那么新粒子 $P_i(t+1)$ 也**必然落在定义域内**，即在迭代过程中，**粒子总是合法、安全的**。
  - 从进化的角度来说，如图所示，相较于$P_i(t)$，新的粒子$P_i(t+1)$确实是**朝着个体极值和全局极值靠近的**。
  - 根据以上分析，无论是从几何角度还是进化角度，式(10-11)都是有意义的。



粒子三角形进化区域（阴影）

4.支撑量优化

$$P_i(t+1) = p \cdot P_i(t) + q \cdot LP_i(t) + r \cdot GP, \qquad (10\text{-}11)$$

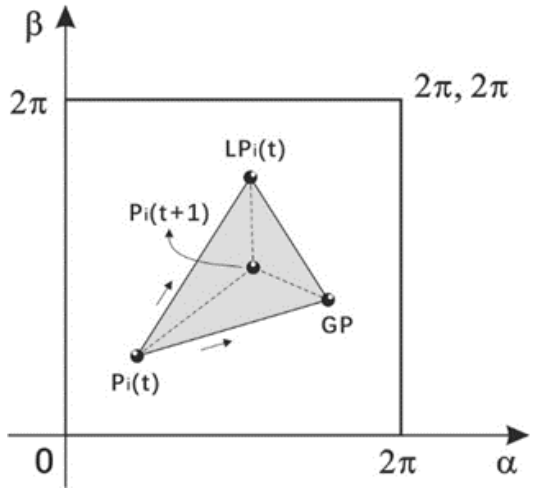
- **4.3. 粒子定义：公式(10-11)的局限性和对策**

口局限性

- 一方面，新的粒子Pi(t+1)最好能被控制在由 Pi(t)、LPi(t)、GP 三点确定的三角形内，以保证粒子在迭代过程中不越界。
- 另一方面，这种控制规律最好不要一成不变，以免迭代中粒子都无法跳出上述三角形，造成**陷入局部最优而无法自拔**的情形。

口应对策略：引入变异操作

- 借鉴遗传算法中基因变异的概念，给定一定概率，**让粒子发生变异**。事实上，从进化角度来说，变异的作用是**制造多样性基因**。

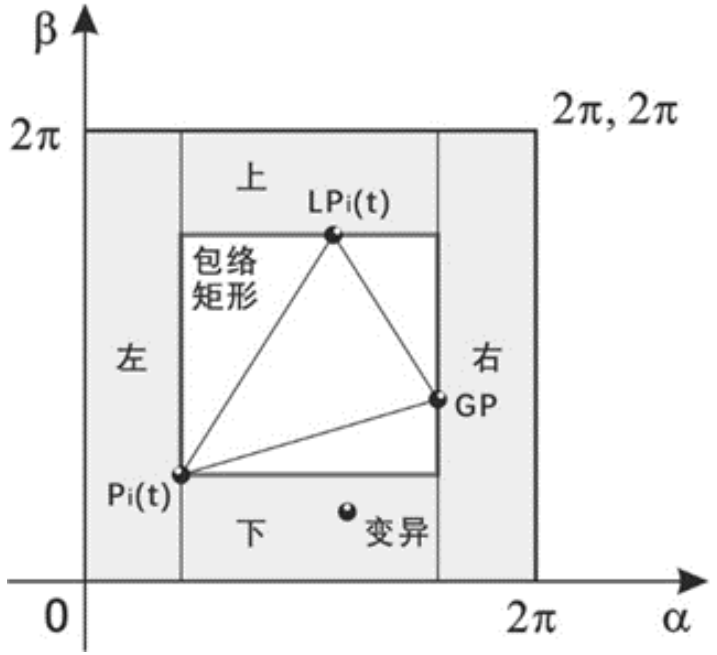


粒子三角形进化区域（阴影）

4. 支撑量优化

$$P_i(t+1) = p \cdot P_i(t) + q \cdot LP_i(t) + r \cdot GP, \qquad (10\text{-}11)$$

- **4.3. 粒子定义：如何变异?**
- 如果要严控控制变异后的粒子落在上述三角形外，直线和三角形边界线的求交（线线求交）操作是不可避免的，这样会让变异操作变得非常繁琐。
- 如图，在不失变异意义的前提下，简单起见，用上述三角形的包络矩形替换三角形本身，并求取落在包络矩形外且在正方形定义域内的随机点作为变异后的粒子。通过这种方式，进行简单的坐标比较便可求出变异后里粒子位置。



粒子三角形进化区域（阴影）

4. 支撑量优化

- **4.3. 粒子定义：代码**

- 以上：①**取消了粒子的速度项，改进了粒子的进化机制**，避免了粒子越界的可能性，②同时**引入了粒子的变异机制**，降低算法陷入局部最优而无法跳出的风险，③同时也给出了粒子适应度的计算函数。
- 下面在 **PsoModelPos** 模块中定义粒子类，类名为 **Particle**，具体实现代码如下：

PsoModelPos.py	定义粒子类，封装拷贝、适应度、变异等函数	10-14

```
1   amax = 2*math.pi                                    # 模型转角取值上限
2   class Particle:                                     # 定义粒子类Particle
3     def __init__(self, a = None, b = None, f = None): # 粒子初始化函数
4       self.a = uniform(0, amax) if a is None else a   # 属性a为模型绕X轴转角
5       self.b = uniform(0, amax) if b is None else b   # 属性b为模型绕Y轴转件
6       self.f = float('inf') if f is None else f       # 属性f为粒子适应度
7     def clone(self):                                  # 复制一个粒子
8       return Particle(self.a, self.b, self.f)
9     def fitness(self, stlModel, gridSize, crAngle):   # 在类中封装适应度函数
10      self.f = fitness(stlModel, self.a, self.b, gridSize, crAngle)  # 调用全局适应度函数
11    def evolve(self, LP, GP, wL, wG):                 # 粒子进化函数
12      r1, r2, r3 = random(), random(), random()
13      sum = r1 + wL * r2 + wG * r3
14      p, q, r = r1 / sum, wL * r2 / sum, wG * r3 / sum  # 根据式(10-13)获得p、q、r
15      self.a = p * self.a + q * LP.a + r * GP.a         # 根据式(10-11)对粒子进化
16      self.b = p * self.b + q * LP.b + r * GP.b
17    def vary(self, prob, LP, GP):                     # 粒子变异函数
18      if random() < prob:                               # 只在指定概率下变异
19        a = uniform(0, amax)                                # 先随机生成转角a
20        if a < min(self.a, LP.a, GP.a) or a > max(self.a, LP.a, GP.a):  # 再根据a确定转角b
21          self.b = uniform(0, amax)                     # 变异点在包络矩形左右两侧
22        else:
23          if random() < 0.5:                            # 变异点在包括矩形上下两侧
24            self.b = uniform(0, min(self.b, LP.b, GP.b))
25          else:
26            self.b = uniform(max(self.b, LP.b, GP.b), amax)
27        self.a = a
```

4. 支撑量优化

• 4.3. 粒子定义：代码分析

□ **Particle** 类的初始化函数包含3个输入参数，**a、b、f**，分别对应类中三个同名同意属性，其中a、b表示模型绕X、Y轴的旋转角度，即α和β，f为例子适应度。

- 在构造粒子实例时，如果外界不输入任何参数，则初始化函数会调用Python random模块中的uniform函数自动在[0, amax]范围内生成两个随机数，并将这两个随机数赋给转角属性a、b，同时对适应度属性f赋浮点数最大值；否则则根据外界输入的a、b、f值构造粒子。
- 注意：以上amax是用来控制转角α、β上限的全局变量（这里值为2π），之前为了方便理解，指定了α、β的定义域为[0, 2π]；在代码实现时，对定义域的上限进一步参数化，即α、β的取值范围为[0, amax]，方便后期更改转角定义域。

□ **Particle** 类还定义了4个函数，分别为 **clone**、**fitness**、**evolve** 和 **vary**。

- **clone** 函数的作用就是根据当前粒子复制一个新粒子对象。
- **fitness** 函数的作用是求取当前粒子的适应度值，它完全依赖于我们在上一小节实现的计算粒子适应度的全局函数。
- **evovle** 函数的作用是根据式(10-11)对粒子进行进化操作，在它的4个输入参数中，LP和GP分别表示个体极值粒子和全局极值粒子，wL和wG分别为进化中个体极值和全局极值相对于当前粒子本身的权重系数（参见式(10-13)）。
- **vary** 函数的作用是按照指定的概率让粒子发生变异，包含3个参数，分别为prop、LP、GP。其中prob为probability缩写，表示当前粒子发生变异的概率，取值为[0, 1]，一般情况下，粒子变异的概率不能设置的太高，否则影响粒子群算法收敛。参数LP、GP仍然表示个体、全局极值粒子，在vary函数中引入这两个粒子的作用是构建如图10.26所示的三角形及其包络矩形，便于我们定位变异后的粒子位置。

4. 支撑量优化

• 4.4. 改进粒子群算法实现：步骤

- 结合上述粒子定义，同时考虑改进的粒子进化机制和新增的粒子变异环节，下面对常规粒子群算法进行改造，给出**改进粒子群算法的基本步骤**，包括：

  - **步骤1.** 随机初始化 N 个粒子的位置 P_i，其中 $i \in [1, N]$；
  - **步骤2.** 根据指定适应度函数计算每个粒子适应度；
  - **步骤3.** 比较粒子群中每个粒子 P_i 和与其对应的个体极值 Lp_i 的适应度，如果 P_i 优于 LP_i，则用 P_i 替换 LP_i；
  - **步骤4.** 比较粒子群中每个粒子 P_i 和全局极值 GP 的适应度，如果 P_i 优于 GP，则用 P_i 替换 GP；
  - **步骤5.** 对粒子群中每个粒子按指定的概率**进行变异操作**，并**根据公式(10-11)更新每个粒子的位置**；
  - **步骤6.** 如果停止条件满足，则跳出循环，否则跳至步骤2。

4. 支撑量优化

- **4.4. 改进粒子群算法实现：代码**

- 根据以上步骤，最后在 **PsoModelPos** 模块中给出改进粒子群算法的实现函数，函数名为 **pso**，具体代码如下：

PsoModelPos.py	取消速度项、增加变异项的改进粒子群算法实现函数	10-15

```
1   ……                                          # 导入相关库，忽略其他代码
2   def pso(stlModel, gridSize, crAngle, popu, iter, prob, wL, wG):      # 粒子群优化函数
3     P, LP, GP = [], [], Particle()            # 定义三种粒子
4     for i in range(popu):                     # 步骤1，初始化粒子
5       P.append(Particle())
6       LP.append(P[i].clone())
7     for k in range(iter):                     # 进入迭代流程
8       for i in range(popu):                   # 步骤2，计算粒子适应度
9         P[i].fitness(stlModel, gridSize, crAngle)
10      for i in range(popu):                   # 步骤3，替换个体极值
11        if P[i].f < LP[i].f:
12          LP[i] = P[i].clone()
13      for i in range(popu):                   # 步骤4，替换全局极值
14        if P[i].f < GP.f:
15          GP = P[i].clone()
16      for i in range(popu):                   # 步骤5，变异和进化
17        P[i].vary(prob, LP[i], GP)            # 粒子变异
18        P[i].evolve(LP[i], GP, wL, wG)        # 粒子进化
19      if round(GP.f, 3) == 0: break           # 如果支撑量接近0，则退出
20    return GP                                 # 返回全局最优粒子
```

必须使用Particle类的clone函数创建需要替换的粒子，而不能直接赋值替换，否则极值粒子将无法正确保存。

大道至简！

4. 支撑量优化

• 4.4. 改进粒子群算法实现：代码分析

- Pso 函数包含8个必要输入参数：
  - **stlModel** 是待优化的目标模型，
  - **gridSize** 为网格投影尺寸，
  - **crAngle** 为危险面片临界角
  - **Popu** 表示粒子群中种群数量，即普通粒子的数量；
  - **Iter** 表示迭代次数；
  - **Prob** 为粒子发生变异的概率；
  - **wL和wG** 分别表示个体极值和全局极值在迭代进化中的权重系数。

这 3 个参数和支撑量优化业务相关，最终将被输入到FindSptRegion中；

这 5 个参数其实和业务无关，是粒子算法本身的参数。

- 在前面工作的基础上，**pso** 函数已非常简单，完全遵循上述改进粒子群的步骤。

- **迭代结束条件：**在上述迭代过程中，如果当前全局极值的适应度已经接近0，即**支撑量约等于0**时，则提前结束迭代过程；否则按设置的**迭代次数**结束迭代。

  - 一般情况下，**严格意义的最优解是未知且无法辨识的**，找到的最优解极有可能只是局部最优解！
  - 对当前的三维打印支撑量优化问题来说，**存在一个特例，那就是支撑量为0**，这个全局最优解是可以被辨识的，但并非所有的三维模型都能达到0支撑量。

4. 支撑量优化

- **4.4. 改进粒子群算法实现：设计思路**

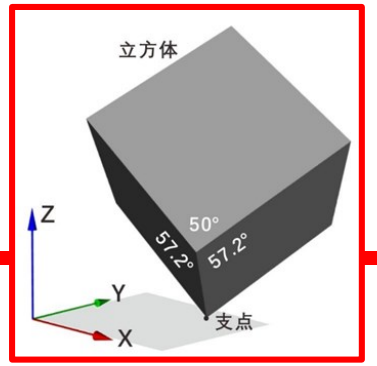
目标

对粒子群算法的实现函数 pso，最好能够给出关于（改进）粒子群算法的框架性代码，具有一定的普适性，这些代码不需要改造或稍微改造一下就能被移植，适用于新的应用场景。

在以上pso函数中正是这么做的！

- ✓ 将和三维打印支撑量优化问题相关的业务逻辑全部封装在 **Particle** 类中，包括适应度函数，进化和变异机制等。
- ✓ 而在 **pso** 函数中，仅保留和（改进）粒子群算法相关的核心步骤和流程。
- ✓ 当然，由于参数传递的原因，在 **pso** 函数中，仍然保留了 stlModel、crAngle 等业务参数，这是难以避免的。

4. 支撑量优化



• 4.5. 支撑优化测试

- **测试模型cube**　边长为150mm，仅有12张三角面片包围而成

- 该立方体仅有一个顶点和XY平面接触，立在XY平面上方，各面（包括对面）相对于XY平面的锐角夹角分别约为：50度、57.2度、57.2度，
- 该模型特别之处在于其**最优支撑量是明确的（0）**，且达到最少支撑量时立方体底面和XY平面平行。
- 方便起见，将对PsoModelPos模块的测试代码写在模块的**main**函数中，具体如下：

PsoModelPos.py	改进粒子群算法测试	10-16

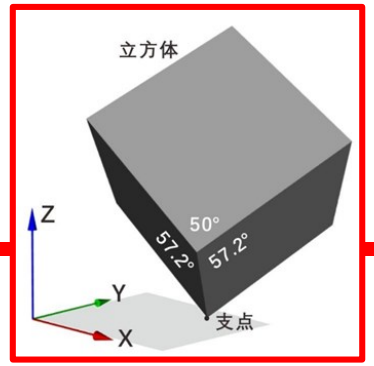
```
1  if __name__ == '__main__':
2      src, stlModel = vtk.vtkSTLReader(), StlModel()
3      src.SetFileName("E:\\Models\\STL\\cube.stl")              # 读取STL模型文件
4      stlModel.extractFromVtkStlReader(src)                     # 提取VTK数据源
5      best = pso(stlModel, 10, degToRad(60), 70, 1000, 0.01, 4, 4)      # 粒子群优化模型摆放角度
6      print("best particle:", radToDeg(best.a), radToDeg(best.b), best.f)  # 打印最优粒子
7      optModel = stlModel.rotated(best.a, best.b, 0)            # 根据优化角度旋转模型
8      layers = slice_topo(optModel, 2.5)                        # 对模型切片
9      findSptRegion(optModel, layers, 2, degToRad(60))          # 生成支撑区域轮廓
10     va = VtkAdaptor()                                         # 以下为显示代码
11     actor = va.drawPdSrc(src)                                 # 显示模型
12     actor.RotateWXYZ(radToDeg(best.a), 1, 0, 0)               # 对VTK场景中的模型按照
13     actor.RotateWXYZ(radToDeg(best.b), 0, 1, 0)               # 优化角度角度旋转
14     for layer in layers:                                      # 显示切片数据
15         for poly in layer.contours:                           # 显示模型轮廓，黑色
16             va.drawPolyline(poly). SetColor(0, 0, 0)
17         for poly in layer.sptContours:                        # 显示支撑轮廓，蓝色
18             va.drawPolyline(poly).SetColor(0, 0, 1)
19     va.display()
```

gridSize = 10 mm

生成支撑区域

显示模型切片和支撑切片

4. 支撑量优化



• 4.5. 支撑优化测试：pso 函数参数设置

□ 业务相关部分参数：

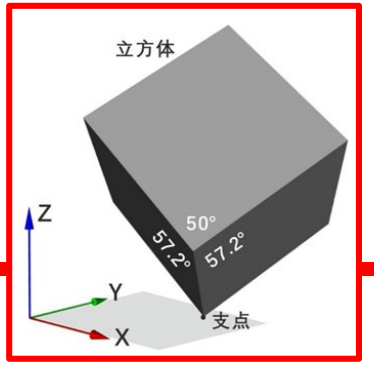
- 考虑到测试立方体边长为150mm，为了节省粒子群算法计算量，设置的模型底面投影网格尺寸（gridSize）为10mm，该设置最终可生成约200个规模的细分网格（参见图10.6和10.7）；
- 设置的危险面片临界角为60度，对测试立方体，初始状态下，它的三个下表面均为危险面，需要添加支撑。

□ 粒子群算法本身相关的5个参数：

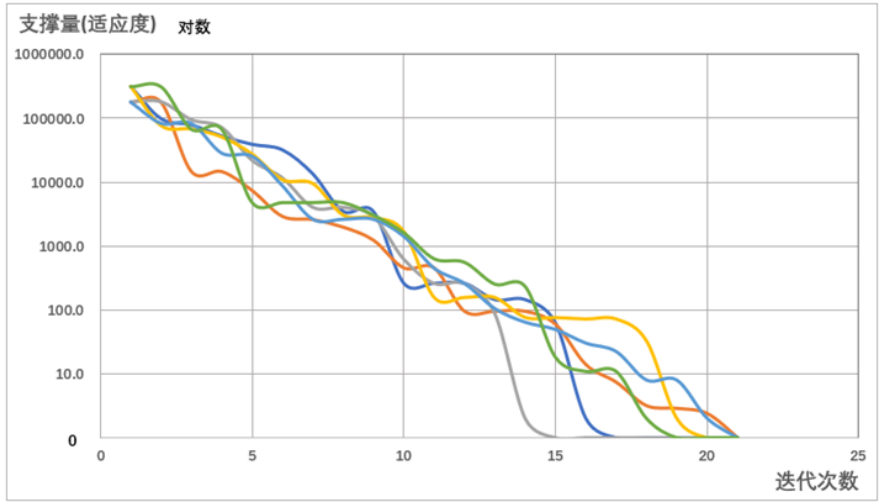
- 设置的粒子种群数量（popu）为70
- 迭代次数（iter）为1000
- 粒子变异概率（prob）为0.01，即一个粒子在一次迭代中有1%的概率发生变异
- 个体极值和全局极值在进化中的权重系数（wL、wG）均为4，即对式(10-11)，p : q : r = 1 : 4 : 4。

□ 需要说明的是，这里设置了较大的迭代次数，希望在算法最大迭代次数到来前能够找到问题的全局最优解。

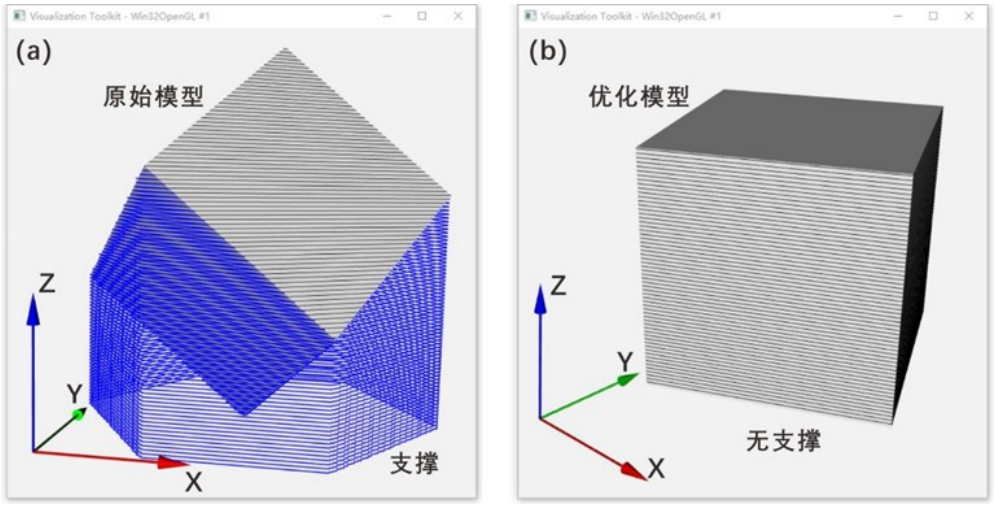
4. 支撑量优化



- **4.5. 支撑优化测试：pso 运行结果**



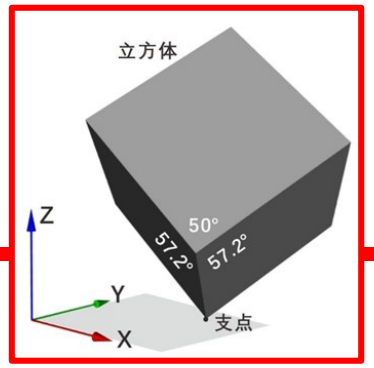
立方体模型粒子群优化中支撑量和迭代次数变化趋势



粒子群算法优化前后立方体模型摆放角度及支撑量变化

- ✓ 左图给出了以上代码多次运行的部分结果统计，其中**横坐标为迭代次数**，**众坐标为粒子适应度（即支撑量）**。可以看到，对立方体模型，粒子群算法具有**立竿见影**的优化效果！
- ✓ 右图显示了优化前后模型摆放角度和所需支撑量的情况。
- ✓ 在测试中，粒子群算法**最少迭代 15 次就能找到问题的全局最优解**，将支撑体积从百万立方毫米级别降到0，这对实际三维打印中减少支撑材料是极有意义的。

4.支撑量优化



• 4.5. 支撑优化测试：结果分析

- 对立方体模型，尽管几次测试中粒子群算法找到的最优支撑量都为0，但**最优解却不尽相同（多解）**，如转角对$(\alpha, \beta) = (232.5°, 207°)$、$(180°, 130°)$、$(52.5°, 153°)$等都是问题的全局最优解。
- 这是可以理解的，考虑到α和β的定义域为$[0, 2\pi]$，在宽松的定义域下，模型就可通过多种方式旋转到最优位置，如对以上最优解$(180°, 130°)$，可以推断，$(180°, 310°)$也必然是问题的最优解。
- 事实上，可以通过缩小α和β的取值范围来减少最优解数量，如让其在$[0, \pi/2]$内取值；但这样会带来另一个问题，如果问题最优解恰在定义域边界上取到，那粒子群在进化、变异过程中就可能“处处碰壁”，不利于最优解确定，这也正是将α、β的定义域设置为$[0, 2\pi]$的原因。
- 以上测试中，对包含12个面片的STL模型，当种群数量为70时，**每次迭代时间约为1.3秒，算法平均收敛时间约为半分钟**，测试主机CPU为intel Core i7-8700 @ 3.20GH。

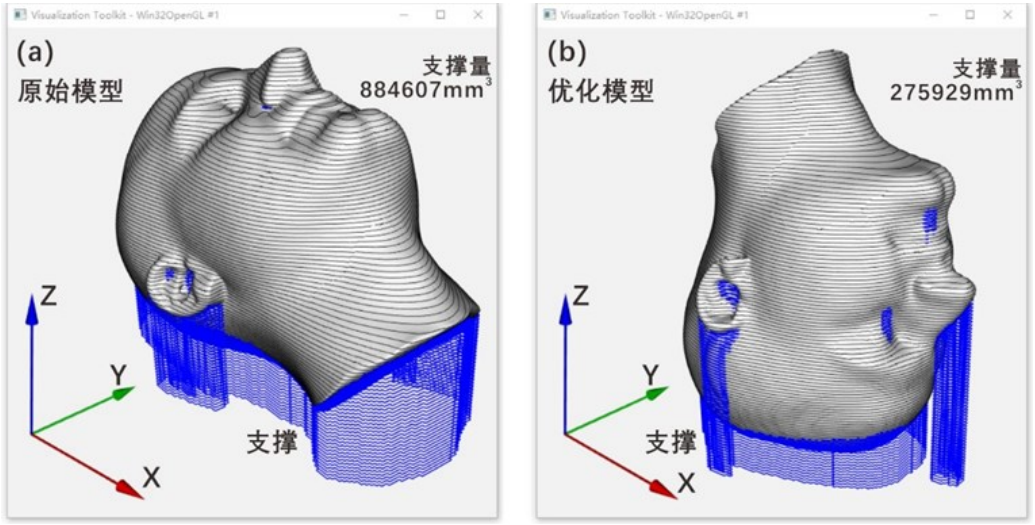
说明：

- 以上只展示了迭代次数相对较少的运行结果。在实际测试中，偶尔也会碰到粒子群迭代上百次仍未收敛的案例。显然，在这些案例中，粒子群陷入了局部最优而无法轻易跳出。
- 由于粒子变异机制的引入，一般情况下，只要再增加迭代次数，这些案例最终也都能收敛并能找到全局最优解。

4. 支撑量优化

- **4.5. 支撑优化测试：测试模型 man head**

- 模型长、宽、高尺寸分别约为 200、200、300mm，共包含45636个三角面片
- 无法预计头像模型最少支撑量数值！！
- 考虑头像模型面片数量大幅增加，同时尺寸上也有所增加，为尽量降低适应度函数计算量，在pso函数中，设置gridSize值为20mm，设置粒子种群数量popu为50，迭代次数（iter）为30。针对以上参数设置，每次迭代时间约为**38秒**。下图清晰地展示了粒子群算法优化前后模型摆放角度和所需支撑量的变化情况。

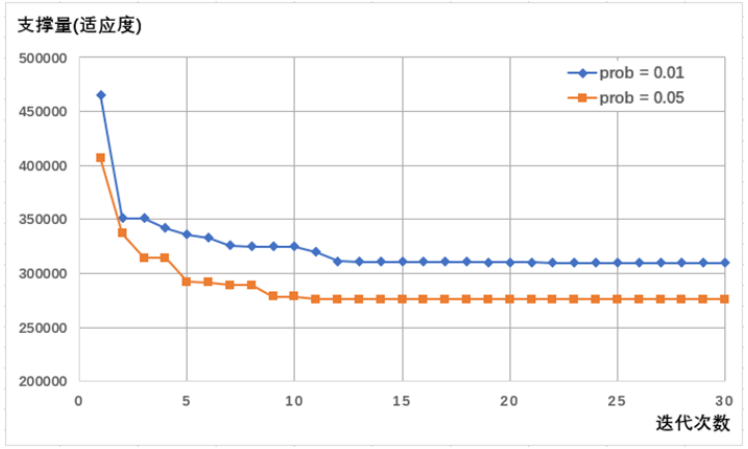


粒子群算法优化前后头像模型摆放角度及支撑量变化

4.支撑量优化

- **4.5. 支撑优化测试：变异的影响（测试模型man head）**

- 对头像模型进行了2组测试，一次设置粒子**变异概率**为0.01，一次设置粒子变异概率为0.05，以检查变异概率对算法寻优的影响。当粒子变异概率为0.05时，粒子群算法能够找到相对更优解。
  - 这是可以理解的，如前所述，变异制造了种群基因的多样性，合理的变异有利于种群向更优的方向进化。
  - 但变异是一把双刃剑，对具体问题应具体分析。事实上，对类似于粒子群算法的优化算法，如何调参以降低计算量、让算法尽快收敛本身就是一门非常学问。
- 对图中的两条曲线，无论是那条曲线，在迭代了约15次以后，曲线就已逐渐趋于平坦，表明此时算法已**找到局部最优解**。
- 从这两条曲线还可以间接看出，在迭代开始前的**初始种群品质对算法寻优是有影响的**，初始种群中的粒子越优秀，算法更容易找到较优解，这显然也是有明确的生物学解释的。



粒子群算法优化前后头像模型摆放角度及支撑量变化

4. 支撑量优化

- **4.5. 支撑优化测试**

小结

- ✓ 经过以上测试，对于三维打印支撑优化这一问题，提出并实现的改进粒子群算法确实是有效的，能够找到最优模型摆放角度，降低所需支撑量。
- ✓ 但该算法在使用过程中仍存在一些问题，如**变异概率**、**种群数量**、**权重系数**等参数设置等，这些参数设置直接影响粒子群算法运行效率和收敛速度。
- ✓ 目前，粒子群算法应用的最大障碍其实是算法本身**计算量过大**，特别是对于那些面片数量较大的三维模型，这主要是由适应度函数计算量过大造成的。
- ✓ 可针对适应度函数的**效率优化**或**粒子适应度**的重新选择等方面**展开进一步探索**。

本章小结

- 这一章主要介绍了面向三维打印的模型支撑生成与优化方法。
  - 首先对三维打印支撑生成进行了概述，提出了支撑空间、危险面片临界角等概念。
  - 基于此，介绍一种生成切片支撑区域的方法。该方法避开直接对支撑空间切片的难题，通过模型底面网格化、模型与网格竖线求交、支撑线段奇偶提取、平面支撑点生成、平面支撑点扩张融合、轮廓偏置、支撑区域布尔运算等步骤及手段生成平面支撑区域。
  - 对得到的切片支撑区域，通过对GenDpPah模块的升级，提出了一种支撑路径的稀疏填充方法。
  - 最后，为了降低支撑材料消耗量，介绍了一种基于粒子群算法的模型摆放角度优化方法。为了限定粒子活动范围，同时增加粒子跳出局部最优的可能性，对粒子群算法进行了改造，设计了一种取消速度项的粒子进化和变异机制。
- 针对上述方法，封装了**FindSptRegion**、**GenSptPath**以及**PsoModelPos**三个模块，实现了findSptRegion、genSptPath以及pso等全局接口函数，通过这些模块和函数，可以对任意STL模型的摆角进行优化，同时生成支撑打印路径。





思考题

1. 设计一种算法，利用STL模型的拓扑关系，提取所有危险面片并将其在各局部区域内连接成片。
2. 在支撑区域外围添加轮廓路径的作用是什么？是否可以取消？
3. 尝试设计一组工艺实验，探索某种常用FDM打印材料（如PLA、ABS等）的支撑临界角度。
4. 简述由输入STL模型生成切片支撑区域的流程。投影网格、模型支撑点、切片支撑点分别是怎么计算得到的？它们的作用是什么？
5. 在由切片支撑点生成切片支撑区域的过程中，为什么使用矩形而非其他图形替换切片支撑点？为什么设置矩形的尺寸略大于投影网格尺寸？
6. 在某一层切片上，如何消除支撑区域和模型区域相互干涉的问题？
7. 对支撑区域的平行填充路径，为什么上下层路径之间最好对齐？如果不对齐会有什么影响？
8. 简述粒子群优化算法的基本流程，简述粒子位移、粒子速度以及适应度函数的概念，说明粒子群优化算法实现问题解优化的本质。
9. 在使用粒子群算法优化模型支撑量过程中，为什么要取消粒子速度项，增加粒子的变异操作？
10. 对模型支撑量优化问题，如何进一步降低粒子群算法中适应度函数的计算量？
11. 尝试使用其他优化算法优化模型支撑量，如遗传算法、模拟退火算法等，并比较这些算法的异同点。